# 运筹优化常用模型、算法及案例实战

## Python+Java实现

刘兴禄　主编
熊望祺　臧永森　段宏达　曾文佳　陈伟坚　编著

清华大学出版社
北京

## 内 容 简 介

本书主要讲述运筹优化领域常用的数学模型、精确算法以及相应的代码实现。首先简要介绍基本理论，然后用丰富的配套案例讲解多个经典的精确算法框架，最后结合常用的优化求解器（CPLEX 和 Gurobi）说明如何用 Python 和 Java 语言实现书中提到的所有精确算法。

全书共分 3 部分。第 I 部分（第 1~4 章）为运筹优化常用模型及建模技巧。该部分着重介绍整数规划的建模技巧和常见的经典模型。第 II 部分（第 5~7 章）为常用优化求解器 API 详解及应用案例。该部分主要介绍两款常用的商业求解器（CPLEX 和 Gurobi）的使用方法，包括 Python 和 Java 的 API 详解、简单案例以及复杂案例。第 III 部分（第 8~17 章）为运筹优化常用算法及实战。该部分详细介绍几个经典的精确算法的理论、相关案例、伪代码以及相应的代码实现。

本书适合作为高等院校工业工程、管理科学与工程、信息管理与信息系统、数学与应用数学、物流工程、物流管理、控制科学与工程等开设运筹学相关课程的高年级本科生、研究生教材，同时也可供在物流与供应链、交通、互联网、制造业、医疗、金融、能源等领域从事有关运筹优化的开发人员以及广大科技工作者和研究人员参考。

**本书封面贴有清华大学出版社防伪标签，无标签者不得销售。**

**版权所有，侵权必究。举报：010-62782989，beiqinquan@tup.tsinghua.edu.cn。**

**图书在版编目（CIP）数据**

运筹优化常用模型、算法及案例实战：Python+Java 实现/刘兴禄主编. —北京：清华大学出版社，2022.9（2023.1 重印）

ISBN 978-7-302-60014-5

Ⅰ. ①运… Ⅱ. ①刘… Ⅲ. ①人工智能 Ⅳ. ①TP18

中国版本图书馆 CIP 数据核字(2022)第 028166 号

**责任编辑：** 刘向威 常晓敏
**封面设计：** 文 静
**责任校对：** 郝美丽
**责任印制：** 丛怀宇

**出版发行：** 清华大学出版社

网 址：http://www.tup.com.cn，http://www.wqbook.com
地 址：北京清华大学学研大厦 A 座　　邮 编：100084
社 总 机：010-83470000　　邮 购：010-62786544
投稿与读者服务：010-62776969，c-service@tup.tsinghua.edu.cn
质 量 反 馈：010-62772015，zhiliang@tup.tsinghua.edu.cn
课 件 下 载：http://www.tup.com.cn，010-83470236

**印 装 者：** 三河市龙大印装有限公司
**经 销：** 全国新华书店
**开 本：** 185mm×260mm　　**印 张：** 35.25　　**字 数：** 839 千字
**版 次：** 2022 年 10 月第 1 版　　**印 次：** 2023 年 1 月第 2 次印刷
**印 数：** 1001~1800
**定 价：** 128.00 元

**产品编号：** 091090-01

# 编 委 会


编委

刘兴禄　清华大学清华-伯克利深圳学院，博士研究生

熊望祺　清华大学工业工程系，硕士

臧永森　清华大学工业工程系，博士研究生

段宏达　清华大学工业工程系，博士研究生

曾文佳　清华大学工业工程系，硕士研究生

陈伟坚　清华大学深圳国际研究生院副院长、清华-伯克利深圳学院副院长、清华大学教授、博士生导师

顾问

戚铭尧　清华大学深圳国际研究生院物流与交通学部副教授、博士生导师

秦　虎　华中科技大学管理学院管理科学与信息管理系教授、博士生导师

# 序1

运筹学（Operations Research）和最优化（Optimization）理论分别植根于管理学和应用数学，都是致力于用数学模型来追求问题的最优决策。工业工程学科追求的也是“复杂系统的系统性最优”，因此，运筹优化也是工业工程的核心理论基础。除了管理学、应用数学、工业工程、系统工程之外，运筹优化也是计算机科学、控制科学、交通运输工程、军事后勤等学科本科或研究生阶段的重要基础理论课程。

运筹优化在生产制造、交通物流、医疗卫生、电力系统、公共服务等生产与服务领域具有广泛的应用价值。以本人所研究的物流工程与管理领域为例，无论是运输，还是仓储，或是生产制造，都离不开运筹优化方法的应用。例如，在物流运输领域，最优决策问题包括设施选址、网络规划、路径规划，以及运输工具的配置等；在仓储领域，可支持仓库布局规划、货位分配、订单拣选、库存优化、机器人调度等决策问题；在生产制造领域，可解决生产设施布局、高级生产排程、机器调度、物料供应优化等问题。同时，在决策层次上，无论是长期的战略层决策、中期的战术层决策，还是短期的操作层决策，都需要用到运筹优化的方法。

本人从事多年的教学、科研和研究生指导工作，在与学生们的交流和共事中发现，即使是专业背景很好的学生，由于在本科阶段所学深度不够，对一些基础概念的掌握不牢固，或者学习理论知识之后缺乏足够的实践，在对实际问题进行建模与求解时，对一些经典问题的编程求解需要较长的时间来熟悉，一些同学甚至到临近毕业时才真正摸索实现了所研究问题的编程求解。而目前已有的教材和参考书，大多偏重运筹优化的基础理论，缺乏具体的编程指导，参考文献则浩如烟海，知识点分散，学习曲线的增长非常缓慢。

本书作者中的很多人都曾和我一起工作过，他们是一群善于思考、具有追本溯源科学精神的年轻人，有些已经走上了工作岗位，继续从事算法研发工作。他们对初学者的困惑感同身受，因而写出了这本非常系统、实用的运筹优化工具书。本书内容覆盖了运筹学基础理论、常见经典问题的数学模型和建模方法、常用求解器的使用指导，以及常用的整数规划算法和分解算法的基础版代码，如分支切割、分支定价、Dantzig-Wolfe 分解、Benders 分解、拉格朗日松弛方法等，并提供了多种语言代码供参考。值得一提的是，书中涵盖了作者在科研学习中日积月累的很多处理模型的小技巧，例如整数规划中各种非线性项的处理

方法。相信这本书一定能为相关领域的学生、科研工作者和产业界研发人员提供非常“解渴”的帮助。

戚铭尧
清华大学深圳国际研究生院物流与交通学部
副教授　博士生导师
2022 年 3 月

# 序3

运筹学本质上具有较强的学科交叉属性，其理论基础依赖于高等数学、高等代数和数理统计，而实践应用依托于计算机学科，需要编程实现相关的求解算法。理论和实践的双重门槛，使得运筹学的发展受到了一定程度的滞后。运筹学虽然是许多专业的必修课程，但现有教材和教学往往只侧重于理论方面，但当真正应用到实际问题的求解时，系统的参考资料却非常少，学生和从业者们只能借助零散的网上资料不断摸索提升。因此，如何快速实现所需要的算法，往往让初学者无从下手，甚至望而却步。另一方面，随着国内企业的快速发展，面临的问题规模增长迅猛，智能化的调度与决策需求倍增，正迎来国内运筹学的黄金时代，必然将对从业者的动手实践能力提出更高的要求。

《运筹优化常用模型、算法及案例实战》这本书，从运筹学基本概念、模型到算法，循循善诱，用大量的案例分析来解释运筹学中的常用理论。不管是模型还是算法，都采用比较详细的步骤进行阐述，让读者一目了然，尤其对于初学者来说，更加容易领略到所解释模型的精髓。比如列生成方法，是求解许多复杂问题的高效算法，但却不易理解，算法实现更是难上加难。大部分参考书中一般只给出一种例子，但这本书分别用 Cutting Stock、TSP、VRP、UFTP、TP 等问题从多个角度进行了建模、算法设计的分析，理论剖析不可谓不透彻明了。

与此同时，本书不以深挖理论分析为主，而是清晰分析了基础理论之后，用经典案例进行实战分析，给读者清楚明白的代码以帮助理解问题。不论是求解器的调用求解，还是设计算法求解，都难能可贵地给出了完整代码，基于这些框架，读者便可以快速实现自己所研究问题的代码。这对于编程基础薄弱的运筹学学习者来说，是莫大的福音。此外，这本书并没有只局限于比较简单的算法讲解，像复杂的 Branch and Price 和 Dantzig-Wolfe 等分解算法，也进行了分析和实战演示，对于运筹学进阶读者，相信也能够有所收获。

综上，对于这本书，我个人认为其弥补了国内运筹学领域在实战教学方面的教材空白，不论是对于初学者，还是进阶者，都有较大的参考价值，同时相信其对于运筹学的科普也将具有一定作用。

张真真

同济大学经济与管理学院管理科学与工程系

副教授　博士生导师

2021 年 12 月

# 序4

2017 年，还在海德堡大学离散与组合优化实验室读博的我在知乎写下第一篇运筹学综述的文章，当时这个专业在行业外还比较“小众”。全网几乎没有专门针对运筹学的自媒体平台和有影响力的科普社区，相关的业界招聘岗位须知里，对口专业有统计学、计算机，却唯独少了运筹学。几个月后，“运筹 OR 帷幄”应运而生。

运筹学被广泛应用于互联网、供应链、交通、金融、能源等多个行业和领域。2022 年的今天，伴随着人工智能、大数据、物联网、智能制造和智慧交通的“东风”，各行各业的数据（库），这一运筹优化的基本要素，正在逐步通过政府、企业的管理信息系统建立和完善。只有通过数字化将多方数据打通，运筹优化才有其用武之地，方得运筹帷幄。正是得益于数字化这一“新基建”，运筹优化作为机构和企业智能决策的“大脑”，近年来受到了学术界和产业界越来越多的关注，迎来了最好的发展时期。

运筹优化是一门高度交叉的学科，“散落”在数学、工业工程、管理科学、交通工程、物流工程、计算机等院系，涵盖的知识体系广，对数学建模、算法设计和编程能力要求较高。运筹优化是一门“实战出真知”的学科，虽然面向学生教研用的优质教材较多，但它们大都聚焦于晦涩难懂的优化理论和数学证明，缺少面向实际应用的教材，也使得高校培养体系与企业实际需求脱节。

2019 年，在深圳举办的一次“运筹 OR 帷幄”线下活动中，我结识了清华大学的刘兴禄博士。他热爱科普创作，在“运筹 OR 帷幄”发表了几篇质量不错的算法实战类文章。作为志同道合的朋友，我很荣幸推荐他的这本新书《运筹优化常用模型、算法及案例实战》。本书不但用通俗易懂的语言介绍运筹优化，特别是整数规划方向的基本理论，还辅以详尽的案例和代码，使读者能够掌握解决一个实际问题从数学建模、设计到算法实现的全流程，适合应用类学科的研究生和业界运筹优化工程师选用。运筹学正处于蓬勃发展的时期，希望更多的读者通过阅读本书学以致用，为各行各业撒播运筹优化的种子。

沈若冰

海德堡大学数学博士、欧盟玛丽·居里学者、德国无人驾驶资深研发工程师

知乎等多个平台签约自媒体人、B 站视频 UP 主，“运筹 OR 帷幄”技术社区、

“全球留学 DIY 飞跃计划”留学科研社区、

“DeepMatch 火柴交友”硕博交友社区创始人

2021 年 7 月

# 前 言

1. 为什么要写这本书

近年来，国内从事运筹优化学术研究的科研人员和工业界的运筹优化算法工程师日益增多，运筹优化逐渐得到国内各行各业的重视，这也是为广大运筹从业者所喜闻乐见的。物流、交通、供应链、电商、零售业、制造业、航空、金融、能源、定价与收益管理等各个领域，都有大量运筹优化的应用场景，同时，也有不少复杂的实际问题亟待解决。这对于国内从事运筹学研究的学者和算法工程师而言，无疑是巨大的挑战和机遇，对于该领域的在校博士研究生、硕士研究生，甚至本科生而言，亦是如此。

“问渠那得清如许？为有源头活水来。”一个行业要想长期欣欣向荣，就需要源源不断地涌入优质的行业人才，而行业人才最重要的来源，就是在校博士研究生、硕士研究生和本科生。拥有高水平的运筹优化领域的研究生、本科生教育，是培养出优质行业人才的重要条件。打造丰富多样的优质教材是提高一个领域的教育水平的重要举措。我在硕士研究生阶段，一直留心调研国内运筹优化教材的现状，发现到目前为止，市面上面向本科生教育的优质教材比较多，这些教材在基础理论的讲解上做得非常到位。但是，国内市面上面向研究生，甚至是已经从业的运筹优化算法工程师的优质教材并不多见，至于聚焦在有针对性地、详细地介绍运筹优化常用算法及其编程实战的教材，更是屈指可数。目前国内运筹优化领域的研究生教育所采用的高级运筹学教材，也大多使用国外的课本，这些课本虽然在基本理论讲解方面详尽透彻，但往往在实战方面却少有涉及。大部分现有的教材都聚焦在讲解基本概念、基本理论、公式推导等方面，而不涉及具体代码实现层面的细节和技巧。国内的很多教材，也都聚焦在一些晦涩的理论推导及证明上，这让很多初学者望而却步。作为一名已经掌握了一些本科生阶段运筹优化知识的学生或从业者而言，要找到一本合适的进阶版中文教材，以满足科研或者工作的需要，是非常困难的。本书就是一本同时能满足本科生课外拓展、研究生科研需要和从业人员实战需要的教材。

“纸上得来终觉浅，绝知此事要躬行。”算法是一个非常讲究实战的领域，仅仅了解理论，而不能动手将其实现，很多时候并不能真正地掌握算法的精髓，达到融会贯通的境界。对于从事科研工作的硕士研究生和博士研究生，以及业界算法工程师而言，不能编程实现算法，意味着科研工作或者企业项目不能被推进。因此，我认为编写一本既能简洁清楚地讲解基本理论，又能提供非常详细的案例和配套代码的运筹优化算法教材，是非常有必要的。我在硕士研究生阶段的初期，研究课题为车辆路径规划（VRP）。为了推进这个研究，我阅读了大量的文献和教材，找到了论文的创新点，然后自以为可以较快地推进研究。但是，真正着手做研究的时候，发现还需要掌握多方面的知识和技能，包括：第一，熟练掌握至少一种编程语言，这是为了能实现涉及的算法；第二，懂得并且熟练使用整数规划的

各种高级建模技巧；第三，掌握一些常用的精确算法或者启发式算法；第四，掌握至少一个优化求解器的使用方法；第五，将所选算法应用到自己课题的模型中去，并将其完整地实现。这还只是完整地复现一遍，不包括模型创新和算法创新的部分。

整个过程听起来似乎并不很困难，但是执行起来让我倍觉吃力。阅读文献、理解论文主旨和其中的算法原理，这部分比较轻松。可是到了算法实现方面，我遇到了一系列的困难，包括算法细节、数据结构、编程语言等，尤其是后两部分，让我不知所措。虽然我在本科阶段也学过一些编程，但是由于学习并不深入，缺乏针对性的练习，编程能力非常薄弱。正因为如此，我在独立实现这些算法的过程中困难重重。于是我去 github 等平台到处搜集资料，希望能得到一些非常对口的代码或文档，但是在 2016 年和 2017 年，平台上并没有非常相关的高质量参考资料，同类平台上也是类似的情况，资料零零散散，逐个筛选非常费时费力。

一个偶然的机会，我在朋友圈看到了一个微信公众号发布的技术科普文章，名为《分支定界法解带时间窗的车辆路径规划问题》，我滑到文章顶部一看，这个公众号叫“数据魔术师”。令我兴奋的是，该文章提供了完整的代码（源代码作者为华中科技大学黄楠博士）。我兴高采烈地下载了这篇文章的详细代码，如获至宝。之后我仔细研读，反复学习，最终找到了常见的精确算法实现的窍门。

随后的一段时间，我就接连开始攻关分支定界算法、分支定价算法等精确算法，并在课题组组会上展示和交流。我读硕士研究生阶段的导师，清华大学深圳国际研究生院物流与交通学部副教授戚铭尧老师也非常高兴，非常支持我钻研这些算法，在理论和实践层面都给予了我很多指导。尤其是关于算法的一些细节方面，老师跟我有多次深入的讨论，很多困惑也是在这个过程中得到解决的。另外，戚老师课题组内研究物流网络规划、车辆路径规划、智能仓储系统等方向的师兄师姐发表的论文中，也在频繁地使用分支界定算法、分支定价算法和列生成算法等精确算法。总之，经过长时间的认真钻研，我终于大致掌握了上述算法。

后来，在研究 Dantzig-Wolfe 分解和拉格朗日松弛时，我已经快要硕士毕业了。当时我照例去 github 寻找参考资料，并且发现了一段非常高质量的代码。非常幸运，我联系到了这个代码的作者，他叫伍健，是西安交通大学毕业的硕士，现在是杉数科技的算法工程师。在这个高质量代码的帮助下，我很快按照自己的理解完成了上述两个算法的实现。另外，在实现的过程中，他也解答了我在算法细节方面的诸多问题。由于他的代码框架远胜于我的代码框架，所以这部分的代码我就参照他原来代码的大框架重新写了一版，放在本书相应章节。不久前我开始撰写这两章，联系他阐明此事，他欣然同意，并且非常支持我，这让我非常感动。在这里，我也代表本书的作者们以及将来的读者们向伍健表示感谢！

在整个算法的学习过程中，我和同课题组的熊望祺（也是本书的作者之一）经常探讨，从零开始学习这些算法，然后一一实现它们，难度究竟如何？我们的观点非常一致：实属不易，参考资料很少很杂，质量不高！我们不止一次地感叹，为什么没有一本能解决我们大部分疑惑的资料呢？当我们学习理论时，可以很轻松地找到好的参考资料，但是在尝试去实现这些算法时，却难以获取详细的资料。要么就只能钻研数百页的较为复杂的用户手册，自己慢慢筛选有用信息。要么就只能找到一些零散的代码资料和文档，大部分时候这

些代码甚至没有任何注释，就连代码实现的具体模型、具体算法都没有做解释说明，更不用说细节方面的解释了。通常的情况是，读者看懂了 A 模型的算法，经过筛选大量资料，却搜集到了 B 模型的代码，而 B 模型的代码对应的算法、注释很不齐全，每个函数的具体功能也没有提供有用的文档。如果读者要硬着头皮研读，可能会花费大量时间，并且很有可能做无用功。相信经常编程的同行都了解，阅读别人的代码是多么痛苦的事，更何况是没有伪代码、没有注释、没有 README 的代码。

在读博以后，我和本书的另外两名作者（臧永森和段宏达）在闲聊时，常常聊到运筹优化方向参考资料匮乏这件事。慢慢地，我萌生了把这些资料整理成书出版，供想要入门和进阶的博士生、硕士生、本科生或者业界的算法工程师学习、查阅的想法。我也将这个想法告诉了熊望祺，他非常赞成，并表示愿意一起合作将这本书整理出版，于是他将平时完成的部分精确算法的代码提供给了我，并将这些内容整合在本书相应章节中。之后，我也向臧永森和段宏达表达了合作意愿，他们也都认为这是一件有价值的事。不久后，他们也正式加入了编写团队，为本书做出了非常重要的贡献。

读博期间写书需要花费大量时间，个别时候会耽误一点科研进度，因此我非常担心导师不同意我做这件事。但是当我向导师清华-伯克利深圳学院副院长陈伟坚老师表明了我写书的计划以后，陈老师非常支持，并鼓励我多做调研，明确本书的受众，构思好书的脉络，要坚持做完整，不要半途而废。然后多次和我讨论如何设计本书的框架，如何编排各个章节，一些细节相关的内容如何取舍，以及针对不同受众如何侧重，还对本书未来的拓展方向、需要改进完善的资源提出了很多非常有帮助的建设性意见。这些意见对本书的顺利完成起到了不可或缺的作用。

目前正在攻读博士的我，虽然对运筹优化有极大的热情，但是回想起过去两三年前的“小白”阶段的学习过程，仍觉得比较痛苦。很多次，我都心力交瘁，濒临放弃的边缘。我个人觉得，像我一样有同样困惑的同行，兴许有不少吧。我不希望每个像我一样的运筹优化爱好者（初学者），都去经历那样的过程，把大量时间浪费在甄别杂乱、不系统的资料中。我认为，国内应当有一些完整的、系统的资料，帮助运筹优化爱好者们突破入门，走上进阶，把主要精力放在探索更新的理论中，或者将这些理论应用于解决新的问题上，真正创造价值。这也是我们花了大量时间和精力，将一些常用技巧、模型、算法原理、算法的伪代码以及算法的完整代码实现通过系统的整理，以通俗易懂的语言串联在一起，最后编成这本书的初衷。

“不积跬步，无以至千里；不积小流，无以成江海。”我从硕士一年级就开始慢慢探索积累，直到博士三年级，终于完成了全书的撰写。如今这本书得以面世，心中又喜又忧。喜自不必说，忧在怕自己只是一个博士在读生，见解浅薄，功夫远不到家，并且书中一定存在一些自己还没发现的错误，不够完美，影响读者阅读……总之，走过整个过程，才越来越深入理解厚积薄发的含义。回想过去几年，真是感慨万千。我曾一次次在实验室 debug 到凌晨，直到逮到 bug，代码调通，紧锁的眉头顿时舒展。然后我潇洒地将笔记本合上，慢悠悠插上耳机，听着自己喜欢的歌，撒着欢儿，披着学术长廊的灯光，傍着我的影子就溜达回宿舍了。那种成就感和幸福感，着实是一种享受。虽然那段时间我经常宿舍、食堂、实验室三点一线，听起来这种生活似乎毫无亮点，枯燥乏味，但是我每天都在快速进步，我

很喜欢这样的状态。这种状态很像陶渊明先生在《五柳先生传》中的描述："好读书，不求甚解；每有会意，便欣然忘食"。不求甚解在这里不必纠结其具体观点，单就欣然忘食这种喜悦感，我认为是相通的。当自己真正有收获时，内心的快乐是油然而生的。也正由于那段时间的积累，才有了现在这本书。博观约取，厚积薄发，希望低年级的同学们坚持积累，不断提升，最终达到梦想的顶峰！

2. 本书内容安排

为了能够让读者系统地学习运筹优化算法，我们将本书分成以下 3 部分：运筹优化常用模型及建模技巧、常用优化求解器 API 详解及应用案例、运筹优化常用算法及实战。

**运筹优化常用模型及建模技巧**部分不需要读者有任何的编程基础，只需要掌握本科的运筹学线性规划相关理论和基本的对偶理论以及整数规划基础知识即可顺利读懂。本部分分为 4 章。第 1 章介绍了一些数学规划模型的分类和后面章节的精确算法会涉及的一些凸优化领域的概念，包括凸集、凸包等。由于本书的重点不在于此，因此本章介绍得比较简略。第 2 章介绍了一些非常常见的运筹学问题，如指派问题、旅行商问题、车辆路径规划问题和多商品网络流问题。这些问题非常经典，并且具有代表性，当下很多实际问题都可以转化为这些问题的变种和拓展。第 3 章讲解了整数规划中常用的建模技巧，包括逻辑约束的用法、非线性项的线性化方法等，这些技巧频繁地出现在业内顶级期刊发表的论文中，因此本书专门设置了一章介绍这些内容。第 4 章讲解了运筹优化中一个非常重要的理论——对偶理论。不同于其他教材，本书中这一章主要聚焦如何写出大规模线性规划的对偶问题，是作者通过自己探索总结出的方法，目前国内外的各种教材和网站，鲜有看到类似的方法介绍。

第 I 部分旨在为后续的算法部分做铺垫，之后章节中会多次用到这部分介绍的概念和模型。

**常用优化求解器 API 详解及应用案例**部分分为 3 章，主要是介绍常用优化求解器（CPLEX 和 Gurobi）及其应用案例。这部分将为之后的算法实战做好技术铺垫，本书第 III 部分的章节，都需要用到这一部分的内容。要顺利读懂这一部分，需要读者有一定的编程基础，至少需要掌握 Java 或者 Python 中的一门语言。第 5 章详细地介绍了 CPLEX 的 Java 接口的用法。这一章的主要内容是根据 CPLEX 提供的用户手册整理而来，包括基本的类、callback 以及例子库中部分例子代码的解读。该章能够帮助读者快速地掌握 CPLEX 的 Java 接口的使用，读者无须去研读厚厚的英文版用户手册。第 6 章系统地介绍了 Gurobi 的算法框架及其 Python 接口的用法。包括常用类、Python 调用 Gurobi 的完整建模过程、日志信息、callback 等内容。最后还附以简单的案例帮助读者理解。第 7 章提供了带时间窗的车辆路径规划（VRPTW）的代码实现。详细地给出了如何调用求解器建立 VRPTW 的模型并求解。这个案例略微复杂，掌握了这个案例，其余更高难度的案例也就迎刃而解。本章的案例都是直接调用求解器得到模型的解，并没有涉及自己实现精确算法的内容。

**运筹优化常用算法及实战**部分是本书最为重要、干货最多的部分。该部分全面、系统地将运筹优化中常用的精确算法以通俗易懂的方式讲解给读者，尽量避免晦涩的解读。为了方便读者自己实现算法，我们为每一个算法都提供了详细完整的伪代码，这些伪代码可以帮助读者从 0 到 1 动手实现相应的算法。在第 8 章，我们简要回顾了单纯形法，并给出

了伪代码和 Python 代码实现。在第 9 章，我们介绍了求解最短路问题的 Dijkstra 算法及其实现。Dijkstra 算法也是一个非常常用的基础算法。第 10~17 章，我们以理论 + 详细小案例 + 伪代码 + 复杂大案例 + 完整代码实现的方式，为读者介绍了分支界定算法、分支切割算法、列生成算法、动态规划算法、分支定价算法、Dantzig-Wolfe 分解算法、Benders 分解算法、拉格朗日松弛 8 个经典且常用的精确算法。这些算法经常出现在运筹学领域各个期刊的文章中以及工业界的具体项目之中。为了便于读者理解，我们尽量避免复杂的数学推导，着重讲解基本原理和算法迭代步骤，真正意义上帮助读者从理论到实践，一步到位，无须到处寻找零散资料，做重复性的整合工作。

相信认真研读这本教材的读者，一定会大有收获。尤其是刚入门的硕士生和博士生，可以凭借这本教材，系统地了解本领域的精确算法，更好地胜任自己的科研工作。对于已经从业的运筹优化算法工程师，本书也可以作为一本非常详尽的学习工具书。

这里需要做一点特别说明，本书不同章节代码的继承性不大，大部分章节的代码都是针对该章节独立编写的。这种做法是为了方便初学者较快地理解每一章的代码，更好地消化每个单独的算法。但是这种编排也有一些弊端，即不利于拓展和改进。实际上，大部分精确算法之间都是有联系的，科研过程中也经常将它们组合使用，如果将所有章节的代码做成一个集成的算法包，既方便管理，又便于拓展。但是，集成性较好的代码，其结构一般都很复杂，初学者面对这样的代码，往往不知所措。复杂的函数调用关系，众多的类和属性，难免会让初学者望而生畏。基于此，本书各个章节的代码还是保持了较高的独立性，今后如果有机会，我会考虑提供两个版本的代码，即独立版本和集成版本，前者主要面向初学者，后者主要面向较为熟练的读者。

为了方便读者更容易地对照本领域内的英文文献，本书对涉及的专业名词（概念、问题名称和算法名称等）给出了中英文对照表。代码手册可到清华大学出版社官网下载。

由于作者水平和时间有限，书稿在编写的过程中，难免有疏漏和错误，一些源自网络的参考资料也由于时间久远不能找到出处，希望涉及的原作者看到以后，及时与作者联系。再版之际，我们将会做出修改。也希望广大热心读者为本书提出宝贵的改进意见。

**3. 本书内容的先修课**

**运筹优化常用模型及建模技巧**部分：先修课程为“运筹学”。另外，最好修过“凸优化”这门课（没有选修这门课也不影响阅读学习）。

**常用优化求解器 API 详解及应用案例**部分：先修课程为“Java 编程基础”或者“Python 编程基础”。

**运筹优化常用算法及实战**部分：修过以上两部分先修课程的同学，可以无障碍阅读学习。如果还修过“高级运筹学”或者“整数规划”，学习会更加轻松。如果没有修过上述课程，也不影响学习，我们的教材内容非常通俗易懂。

**4. 致谢**

非常感谢我读博士研究生阶段的指导老师，即清华大学清华-伯克利深圳学院陈伟坚教授。陈老师对本书整体框架、章节安排等方面提出了诸多建设性意见，也参与了本书的校对、完善等工作。而且，陈老师从读者角度出发，对全书内容的精炼等方面提出了若干宝

贵修改意见，有效地提升了本书的可读性和可拓展性。另外，陈老师对本书后续的完善以及图书习题部分的补充也给出了明确方向。

感谢我读硕士研究生阶段的指导老师，即清华大学深圳国际研究生院物流与交通学部戚铭尧副教授。戚老师对本书算法理论介绍部分以及内容完整性方面提出了诸多宝贵的修改意见。

感谢清华大学深圳国际研究生院物流与交通学部张灿荣教授。张教授是我读硕士研究生阶段的高级运筹学老师，他妙趣横生的讲解激发了我对运筹优化浓厚的学习兴趣。

感谢运筹优化领域微信公众号“数据魔术师”给予我的莫大帮助。该公众号发布的优化算法介绍文章以及完整代码资料为我实现精确算法提供了非常有用的参考。特别地，在 Branch and Bound 的实现过程中，我参考了该公众号分享的部分代码。由衷感谢该公众号的运营者：华中科技大学管理学院秦虎教授以及他的团队。秦虎老师的团队发布的算法科普文章以及举办的精确算法系列讲座，犹如雪中送炭，为很多运筹优化领域的初学者提供了巨大帮助。同时非常感谢华中科技大学的黄楠博士，他提供的源代码帮助笔者克服了诸多困难。

感谢 Mschyns 在 github 上开源的 Branch and Price 求解 VRPTW 的代码，以及华中科技大学邓发珩同学做的修改。该版本的 Java 代码可以为读者自行实现 Branch and Price 算法提供重要参考。

感谢伍健在 DW-分解算法、拉格朗日松弛算法方面提供的资源。非常感激在算法实现过程中，伍健对我诸多疑惑的耐心解答。也非常感谢伍健对这本书出版的大力支持。

此外，我还很荣幸地邀请到了我的老师、师兄、师姐、师弟、师妹们参与了本书后期的读稿和校对工作。他（她）们是陈名华（香港城市大学教授，我的老师）、张莹、黄一潇、陈锐、王祖健、游锦涛、何彦东、王美芹、王梦彤、段淇耀、贺静、张文修、周鹏翔、王丹、夏旸、李怡、郝欣茹、朱泉、修宇璇、王涵民、张祎、梁伯田、陈琰钰、王基光、徐璐、左齐茹仪、张婷、李良涛、赖克凡、曹可欣、金广、席好宁、俞佳莉、陈梦玄。非常感谢他（她）们宝贵的意见和建议。

感谢运筹优化领域公众号“运筹 OR 帷幄”。该公众号发布的高质量的理论介绍文章、讲座预告等给予了我大量的信息，在拓宽视野、了解运筹领域交叉学科的发展现状等方面给了我莫大的帮助。

感谢本书所有参考文献的作者，是你们的研究成果让我学到了许多新的知识，获得了不少新的启发。本书部分内容参考或者翻译自这些文献，在此向这些研究者们致以崇高的敬意。

感谢清华大学出版社对本书出版给予的支持，以及参与本书编校工作的编辑们的辛勤付出。

最后，感谢我的父亲和母亲对我的鼓励和培养，以及对我完成学业的无条件支持和理解。

希望本书能得到广大读者的喜爱，为大家提供有效的帮助。

刘兴禄

2020 年 12 月于清华大学深圳国际研究生院

# 作者贡献

本书所有章节的贡献者如下。

表 0.1　作者贡献

| 章　节 | 贡　献 |
| --- | --- |
| 第 1 章：运筹优化算法相关概念 | 段宏达：凸集、极点、多面体、超平面与半平面等概念介绍 (刘兴禄修改)<br>刘兴禄：几类常见的数学规划模型概念介绍 |
| 第 2 章：运筹优化经典问题数学模型 | 刘兴禄、陈伟坚 |
| 第 3 章：整数规划建模技巧 | 段宏达：逻辑约束、线性化部分<br>刘兴禄：分段函数线性化部分及整章修改 |
| 第 4 章：大规模线性规划的对偶 | 刘兴禄 |
| 第 5 章：CPLEX 的 Java API 详解及简单案例 | 熊望祺：全章撰写<br>臧永森：全章修改<br>刘兴禄：全章修改 |
| 第 6 章：Gurobi 的 Python API 详解及简单案例 | 刘兴禄：2.1 节 ～ 2.8 节, 2.10 节、2.11 节及整章修改<br>臧永森：2.9 节<br>曾文佳：Python 调用 Gurobi 可以求解的问题类型部分的代码和 Gurobi 算法介绍部分 |
| 第 7 章：调用 CPLEX 和 Gurobi 求解 MIP 的复杂案例：VRPTW 和 TSP | 刘兴禄 |
| 第 8 章：单纯形法 | 刘兴禄、陈伟坚 |
| 第 9 章：Dijkstra 算法 | 刘兴禄 |
| 第 10 章：分支定界算法 | 刘兴禄：理论介绍、算法伪代码及 Branch and Bound 代码 (手动实现分支版本) 及整章修改<br>熊望祺：Branch and Bound 代码 (callback 实现版本) |
| 第 11 章：分支切割算法 | 刘兴禄：Cutting plane、Branch and Cut 理论介绍伪代码和 Branch and cut 求解 VRPTW 的 Java 代码，Branch-and-cut 部分内容撰写，以及整章修改<br>曾文佳：Branch and cut 求解 CVRP 和 VRPTW 的理论介绍和 Java+callback 实现 |
| 第 12 章：拉格朗日松弛 | 刘兴禄：理论介绍、伪代码以及修改版本的 Lagrangian Relaxation 求解 Location Transport Problem 的 Python 代码及整章的修改<br>伍键：Lagrangian Relaxation 求解 Location Transport Problem 的 Python 代码 (代码原作者) |
| 第 13 章：列生成算法 | 刘兴禄：理论介绍及相应 Column Generation 的 Java 、Python 版本的代码及整章修改<br>熊望祺：Column Generation 的版本 2 的 Java 代码 |

续表

| 章　节 | 贡　献 |
| --- | --- |
| 第 14 章：动态规划 | 臧永森：第一节理论介绍及相应案例的 Java 代码<br>刘兴禄：Dynamic Programming 求解 TSP 的理论介绍、伪代码和 Python 代码及整章修改<br>刘兴禄：SPPRC 的 Labelling algorithm 的理论介绍、伪代码和 Python 代码<br>刘兴禄：Python 实现用 SPPRC 的 Labelling Algorithm 加 Column Generation 求解 VRPTW 的理论介绍和代码<br>熊望祺：Java 实现大规模 SPPRC 的 Labelling Algorithm 的代码 |
| 第 15 章：分支定价算法 | 刘兴禄：理论介绍、算法框架及伪代码及整章修改<br>熊望祺：Java 实现 Branch and Price 的代码 |
| 第 16 章：Dantzig-Wolfe 分解算法 | 段宏达：理论介绍、具体案例和相应伪代码<br>刘兴禄：部分理论介绍、简单案例的 Python 代码和 CG 结合 D-W Decomposition 的 Python 代码及整章修改<br>伍健：Dantzig-Wolfe Decomposition 求解 Multicommodity Network Flow 问题的 Python 代码 (代码原作者) |
| 第 17 章：Benders 分解算法 | 刘兴禄：理论介绍，伪代码、第一个版本的 Benders Decomposition 的 Java 代码，以及修改版本的 Benders Decomposition 求解 Fixed Charge Transportation Problem 的 Python 代码及整章的修改<br>熊望祺：Fixed Charge Transportation Problem 和 Uncapacitated Facility Location Problem 的 Benders Decomposition 部分的 Java 代码<br>伍健：Benders Decomposition 求解 Fixed Charge Transportation Problem 的 Python 代码 (代码原作者)，Benders Decomposition 求解 Uncapacitated Facility Location Problem 的 Python 代码 (代码原作者) |
| 全书框架、章节安排、内容精炼 | 陈伟坚 |

# 术语缩写对照表

| | |
|---|---|
| **LP** | Linear Programming，线性规划 |
| **IP** | Integer Programming，整数规划 |
| **Pure Integer Programming** | 纯整数规划 |
| **MIP** | Mixed Integer Programming，混合整数规划 |
| **MILP** | Mixed Integer Linear Programming，混合整数线性规划 |
| **QP** | Quadratic Programming，二次规划 |
| **MIQP** | Mixed Integer Quadratic Programming，混合整数二次规划 |
| **QCP** | Quadratically Constrained Programming，二次约束规划 |
| **MIQCP** | Mixed Integer Quadratically Constrained Programming，混合整数二次约束规划 |
| **QCQP** | Quadratically Constrained Quadratic Programming，二次约束二次规划 |
| **MIQCQP** | Mixed Integer Quadratically Constrained Quadratic Programming，混合整数二次约束二次规划 |
| **SOCP** | Second-Order Cone Programming，二阶锥规划 |
| **Second-Order Cone Constraint** | 二阶锥约束 |
| **MISOCP** | Mixed Integer Second-Order Cone Programming，混合整数二阶锥规划 |
| **SDP** | Semi-Definite Programming，半正定规划 |
| **Convex Set** | 凸集 |
| **Extreme Point** | 极点 |
| **Polyhedron** | 多面体 |
| **Hyperplane** | 超平面 |
| **Halfspace** | 半空间 |
| **Corner Point** | 角点（极点） |
| **Convex Combination** | 凸组合 |
| **Convex Hull** | 凸包 |
| **Conv($S$)** | 集合 $S$ 的凸包 |
| **AP** | Assignment Problem，指派问题 |
| **Bipartite Matching** | 二部图匹配 |
| **Hungarian Algorithm** | 匈牙利算法，也叫 Kuhn-Munkres Algorithm |
| **Ford-Fulkerson Algorithm** | Ford-Fulkerson 算法，也叫 Edmonds-Karp Algorithm |
| **Hopcroft-Karp Algorithm** | Hopcroft-Karp 算法，也叫 Hopcroft-Karp-Karzanov Algorithm |
| **SPP** | Shortest Path Problem, 最短路问题 |
| **Maximum Flow Problem** | 最大流问题 |
| **MCNFP** | Minimum Cost Network Flow Problem，最小费用网络流问题 |
| **Unimodular Matrix** | 幺模矩阵 |
| **Integer Matrix** | 全整数矩阵 |
| **Totally Unimodular Matrix** | 全幺模矩阵 |
| **TSP** | Traveling Salesman Problem，旅行商问题或货郎担问题 |
| **Subtour** | 子环路 |
| **SECs** | Subtour Elimination Constraints，子环路消除约束 |
| **MTZ** | Miller-Tucker-Zemlin 约束 |

续表

| | |
|---|---|
| STSP | Symmetric Traveling Salesman Problem，对称旅行商问题 |
| 1-tree | 具有入射到点 1 的两条边和在点集 $\{2,3,\cdots,n\}$ 上的一个树 |
| Column Generation | 列生成 |
| VRP | Vehicle Routing Problem，车辆路径规划问题 |
| CVRP | Capacitated Vehicle Routing Problem，带容量的车辆路径规划问题 |
| VRPTW | Vehicle Routing Problem with Time Windows，带时间窗的车辆路径规划问题 |
| Capacitated VRP with Pick-up and Deliveries and Time Windows | 带取送货和时间窗的车辆路径规划问题 |
| Multiple Depot VRP | 多车场车辆路径规划问题 |
| Multiple Depot VRP with Time Windows | 带时间窗的多车场车辆路径规划问题 |
| Periodic VRP | 周期性车辆路径规划问题 |
| Periodic VRP with Time Windows | 带时间窗的周期性车辆路径规划问题 |
| Vehicle Routing Problem with Pick-up and Deliveries | 同时取送货的车辆路径规划问题 |
| NP-hard | NP 难问题 |
| NP-complete | NPC，即 NP 完全问题，NP 问题中最困难的问题 |
| Branch and Bound | 分支定界算法 |
| Branch and Price | 分支定价算法 |
| Branch and Cut | 分支切割算法 |
| Branch Price and Cut | 分支定价切割算法 |
| MCNF | Multicommodity Network Flow Problem，多商品流网络流问题 |
| Either-Or Constraints | 二选一约束条件 |
| If-Then Constraints | 假设约束条件 |
| Duality Theory | 对偶理论 |
| Robust Optimization | 鲁棒优化 |
| Multilevel Programming | 多层规划 |
| Inner Level | 内层 |
| Bi-Level | 双层 |
| Single Level | 单阶段 |
| Benders Decomposition | Benders 分解 |
| Reduced Cost | 检验数 |
| Binding | 边界 |
| Basic Feasible Solution | 基可行解 |
| Original Variable | 初始变量 |
| Slack Variable | 松弛变量 |
| Surplus Variable | 剩余变量 |
| Complementary Basic Solutions Property | 互补基解 |
| Complementary Optimal Basic Solutions Property | 互补最优基解 |
| CPF | Corner Point Feasible，角点 (极点) 可行 |
| Duality Theorem | 对偶定理 |
| Weak Duality Property | 弱对偶性 |
| Strong Duality Property | 强对偶性 |
| Complementary Slackness Property | 互补松弛性 |
| Dual variable | 对偶变量 |

续表

| | |
|---|---|
| RHS | Right-Hand-Side，右端项 |
| LHS | Left-Hand-Side，左端项 |
| Free | 无约束 |
| Commodity | 商品流 |
| Gomory Cuts | Chvátal-Gomory Cut，Gomory 割 |
| Maximal Cliques Cuts | 最大团割 |
| Minimal Cover Cuts | 最小覆盖割 |
| MIR Cuts | Mixed Integer Rounding Cut，MIR 割 |
| Zero-half cuts | 零半割 |
| Flow Cover Cuts | 流覆盖割 |
| Flow Path Cuts | 流程割 |
| Global Implied Bounds Cuts | 全局隐界割 |
| Lift and Project Cuts | 提升与投影割 |
| Chvátal-Gomory Inequality | Gomory 不等式 |
| Gap | （相对）间隙，差距 |
| Callback | 回调 |
| Legacy Callback | 传统回调 |
| Informational Callback | 参考回调 |
| Query or Diagnostic Callbacks | 查询或诊断回调 |
| Generic Callback | 通用回调 |
| Continuous Problems | 连续问题 |
| LP based Branch and Bound | 基于线性规划的分支定界 |
| Cutting Plane | 割平面 |
| Heuristic | 启发式 |
| Presolving | 预求解，也叫预优化 |
| LP Relaxation | 线性松弛 |
| Prune | 剪枝 |
| Node Selection | 节点选择 |
| Branching | 分支 |
| Model Cleanup and Removal of Redundant Constraints | 清理模型，删除冗余约束 |
| Bound Strengthening | 界限加强 |
| Coefficient Strengthening | 不等式约束的 Chvatal-Gomory 加强 |
| Removal of Fixed Variables | 固定变量的去除 |
| Rounding Bounds of Integer Variables | 整数变量的圆整界限 |
| Strengthen Semi-continuous and Semi-integer Bounds | 加强半连续和半整数界限 |
| Redundancy Detection | 冗余检测 |
| Parallel and Nearly Parallel Rows | 平行和近平行行 |
| Parallel Columns | 平行列 |
| Dominated Columns | 优超列 |
| Aggregate Pairs of Symmetric Variables | 对称变量的聚合对 |
| Probing | 探测 |
| Preferred Branch Direction | 分支偏好参数 |
| DFS | Depth-First Search，深度优先搜索 |

续表

| | |
|---|---|
| BFS | Breadth-First-Search，广度优先搜索 |
| Node | 节点 |
| Current Node | 当前节点 |
| Leaf Node | 叶子节点 |
| Greedy Algorithm | 贪婪算法 |
| LP-Based Greedy Algorithm | 基于线性规划的贪婪算法 |
| DP | Dynamic Programming，动态规划 |
| Minimum Spanning Tree | 最小生成树 |
| Rounding | 圆整 |
| Blind Heuristics | 盲目启发式 |
| Reformulation | 模型改建 |
| RINS | Relaxation Induced Neighborhood Search，松弛诱导邻域搜索 |
| Sub MIP & Recursive Solve | 子混合整数规划和递归方法 |
| Feasibility Pump Heuristic | 泵式缩减启发式方法 |
| Pump Reduce | 泵式缩减 |
| Max Fractional Value | 最大分数值 |
| Shadow Costs | 影子成本，类似 Pseudo Costs |
| Pseudo Costs | 伪成本 |
| Strong Branching | 强分支 |
| Pseudo-Cost Branching for SOS Sets | SOS 集合的伪成本分支 |
| Objective Bounds | 目标值界限 |
| Bound | 界限 |
| UB | Upper Bound，上界 |
| LB | Lower Bound，下界 |
| SOS | Special Ordered Set，特殊顺序集 |
| General Constraints | 广义约束 |
| Overloaded Operators | 重载运算符 |
| IIS | Irreducible Inconsistent Subsystem，不可约不相容子方程组 |
| Lazy Updates | 惰性更新 |
| Simplex Algorithm | 单纯形法 |
| Minimum Ratio Test | 最小比值准则 |
| Primal Problem | 原问题 |
| Primal Simplex | 原始单纯形法 |
| Dual Simplex | 对偶单纯形法 |
| Primal-Dual Algorithm | 原对偶法 |
| Complementary Slackness Condition | 互补松弛条件 |
| Dantzig-Wolfe Decomposition | Dantzig-Wolfe 分解，简称 D-W 分解 |
| Dijkstra Algorithm | Dijkstra 算法 |
| Label Algorithm | 标签算法 |
| Label Setting Algorithm | 标签设定算法 |
| Label Correction Algorithm | 标签修正算法，也称标签校正算法 |
| Bellman-Ford Algorithm | Bellman-Ford 算法 |
| Divide and Conquer | 分而治之 |
| Subproblem | 子问题，简写为 SP |
| Branch and Bound Tree | 分支树，简称为 BB Tree，或者 B&B Tree，或者 BnB Tree |
| Branches | 分支 |

续表

| | |
|---|---|
| Fathomed | 查明或洞悉 |
| Prune by Optimality | 根据最优性剪枝 |
| Prune by Bound | 根据界限剪枝 |
| Prune by Feasibility/Infeasibility | 根据是否可行剪枝 |
| Incumbent | 迄今为止获得的最好可行解，作为最紧的下界（对于最大化问题）或上界（对于最小化问题） |
| Root Node | 根节点 |
| Branching Rule | 分支法则 |
| Subtree | 子树 |
| Bounding | 定界 |
| Best-First-Search | 最好解优先 |
| FIFO | First in First Out，先进先出 |
| LIFO | Last in First Out，后进先出 |
| Dominance Relations | 优超关系 |
| Valid Inequality | 有效不等式 |
| Cutting Plane Algorithm | 割平面法 |
| Gomory' s Fractional Cutting Plane Algorithm | Gomory 割平面法 |
| BQP Cut | Boolean Quadric Polytope Cut，BQP 割 |
| GUB Cover Cut | Generalized Upper Bound Cover Cuts，广义上界覆盖割平面 |
| Implied Bound Cut | 隐界割 |
| MIP Separation Cut | 混合整数规划分割割平面 |
| Mod-k Cut | k-模割平面 |
| Network Cut | 网络割平面 |
| Projected Implied Bound Cut | 投影界割平面 |
| Relax-and-Lift Cut | 松弛提升割平面 |
| RLT Cut | Reformulation Linearization Technique Cut，重构线性技术割平面 |
| Strong-CG Cut | Strong Chvátal-Gomory cut，强 Gomory 割平面 |
| Sub-MIP Cut | 子混合整数规划割平面 |
| Branch-and-Cut Tree | 分支切割树 |
| Cut Pool | 割平面池 |
| Robust Cuts | 鲁棒割平面 |
| K-path Cuts | K 路径割平面 |
| Rounded Capacity Cuts | 容量圆整割平面 |
| Non-robust Cuts | 非鲁棒割平面 |
| SRCs | Subset Row Cuts，子集行割平面 |
| Limited-memory SRCs | 有限记忆子集行割平面 |
| Elementary Cuts | Strengthened Capacity Cuts, 容量增强割平面 |
| k-cycle Elimination Cuts | k 环消除割平面 |
| Strong Degree Cuts | 强度割平面 |
| Lazy Constraint | 惰性约束 |
| User Cut | 用户自定义割平面 |
| Cutting Stock Problem | 下料问题，简称 CSP |
| Master Problem | 主问题，可简写为 MP |
| Pricing Problem | 定价问题，又称定价子问题，子问题 |
| RMP | Restricted Master Problem，限制性主问题 |
| Degree | 度 |
| Set Partitioning Problem | 集分割问题 |

续表

| | |
|---|---|
| Set Covering Problem | 集覆盖问题 |
| Cycle | 环 |
| SPPRC | Shortest Path Problem with Resource Constraints，带资源约束的最短路问题 |
| ESPPRC | Elementary Shortest Path Problem with Resource Constraints，带资源约束的基本最短路问题 |
| Dominance Rule | 优超准则 |
| Source | 起点，同 origin |
| Origin | 起点，同 source |
| Sink | 终点，同 destination |
| Destination | 终点，同 sink |
| SPPTW | Shortest Path Problem with Time Windows，带时间窗和资源约束的最短路问题 |
| REFs | Resource Extension Functions，资源扩展函数 |
| Resource Feasible Paths | 资源可行路径 |
| Resource Intervals | 资源间隔，也叫资源上下限，资源窗口等 |
| Resource Windows | 资源窗口，即资源间隔 |
| Path-structural Constraints | 路径结构约束 |
| Elementary Path | 基本路径，即路径中无环路 |
| Acyclic Graphs | 无环图 |
| SPPRC-k-Cyc | SPPRC with k-cycle Elimination，k 环消除约束的 SPPRC |
| SPPRCFP | SPPRC with Forbidden Paths，带禁止路径的 SPPRC |
| Precedence Constraints and Pairing Constraints | 针对送取货的路径结构约束 |
| Underlying Network | 基础网络 |
| Acyclic Time-Space Network | 无环时空网络 |
| Pareto-Optimal | 帕累托最优 |
| Dominance Algorithms | 优超算法 |
| Nearest Neighbor Algorithm | 最近邻居法，简称 NN |
| Saving Heuristic | 节约算法 |
| I1 | Solomon 提出的插入算法 I1 |
| I2 | Solomon 提出的插入算法 I2 |
| Block-angular Model | 块角模型 |
| Minkowski's Representation Theorem | Minkowski's 表示定理 |
| Phase I/II Algorithm | 两阶段法 |
| Central Constraint | 中央约束 |
| Convex Constraint | 凸约束 |
| Coefficient | 系数 |
| Node-arc Model | 点弧模型 |
| MCTP | Multi-Commodity Transportation Problem，多商品流运输问题 |
| Convex Combination Constraints | 凸组合约束 |
| Row Generation | 行生成 |
| CCG | Column and Constraint Generation，列与约束生成 |
| Benders feasibility cut | Benders 问题可行的必要条件 |
| Benders optimality cut | Benders 基于子问题的最优性条件 |
| FCTP | Fixed Charge Transportation Problem，固定费用运输问题 |
| UFLP | Uncapacitated Facility Location Problem，无容量限制的设施选址问题 |
| Lagrangian Relaxation | 拉格朗日松弛 |

续表

| | |
|---|---|
| **Relaxation** | 松弛 |
| **Optimality** | 最优性 |
| **COP** | Combinatorial Optimization Problem，组合优化 |
| **Primal Bounds** | 原始边界 |
| **Dual Bounds** | 对偶边界 |
| **Weak-Dual Pair** | 弱对偶对 |
| **Strong-Dual Pair** | 强对偶对 |
| **Lagrangian Dual Problem** | 拉格朗日对偶问题 |
| **Lagrangian Relaxation Subproblem** | 拉格朗日松弛子问题 |
| **Lagrange Multiplier** | 拉格朗日乘子 |
| **Shadow Price** | 影子价格 |
| **Dual Price** | 对偶价格，即边际收益或影子价格 |
| **Gradient** | 梯度，即导数 |
| **Subgradient** | 次梯度 |
| **Subgradient Algorithm** | 次梯度算法 |
| **GAP** | Generalized Assignment Problem，广义指派问题 |
| **Location Transport Problem** | 选址-运输问题 |

# 目 录

## 第 I 部分 运筹优化常用模型及建模技巧


第 1 章 运筹优化算法相关概念 ····· 3
1.1 几类常见的数学规划模型 ····· 3
1.1.1 线性规划 ····· 3
1.1.2 混合整数规划 ····· 3
1.1.3 二次规划 ····· 4
1.1.4 二次约束规划 ····· 4
1.1.5 二次约束二次规划 ····· 4
1.1.6 二阶锥规划 ····· 5
1.2 凸集和极点 ····· 6
1.2.1 凸集 ····· 6
1.2.2 极点 ····· 7
1.3 多面体、超平面与半空间 ····· 7
1.3.1 多面体 ····· 7
1.3.2 超平面与半空间 ····· 7
1.4 凸组合和凸包 ····· 8
1.4.1 凸组合和凸包的概念 ····· 8
1.4.2 一些结论 ····· 8
第 2 章 运筹优化经典问题数学模型 ····· 9
2.1 指派问题 ····· 9
2.2 最短路问题 ····· 11
2.3 最大流问题 ····· 12
2.3.1 问题描述 ····· 12
2.3.2 问题建模及最优解 ····· 13
2.3.3 最大流问题的一般模型 ····· 14
2.3.4 Ford-Fulkerson 算法求解最大流问题 ····· 15
2.3.5 Java 实现 Ford-Fulkerson 算法求解最大流问题 ····· 18
2.4 最优整数解特性和幺模矩阵 ····· 23
2.4.1 指派问题的最优解特性验证 ····· 24

2.4.2 最短路问题的整数最优解特性验证 ····· 27
2.4.3 最优整数解特性的理解 ····· 31
2.4.4 幺模矩阵和整数最优解特性 ····· 32
2.5 多商品网络流问题 ····· 34
2.6 多商品流运输问题 ····· 37
2.7 设施选址问题 ····· 38
2.8 旅行商问题 ····· 39
2.8.1 TSP 建模方法 1：子环路消除约束 ····· 41
2.8.2 TSP 建模方法 2：MTZ 约束消除子环路 ····· 42
2.8.3 TSP 建模方法 3：1-tree 建模方法 ····· 45
2.9 车辆路径规划问题 ····· 47
2.9.1 概述 ····· 47
2.9.2 VRPTW 的一般模型 ····· 48
**第 3 章 整数规划建模技巧** ····· 51
3.1 逻辑约束 ····· 51
3.1.1 两个命题 ····· 51
3.1.2 二选一约束条件 ····· 51
3.1.3 指示约束条件 ····· 53
3.2 线性化 ····· 54
3.2.1 分段线性函数线性化 ····· 54
3.2.2 含绝对值形式的线性化 ····· 58
3.2.3 含乘积形式的线性化 ····· 60
3.2.4 含分式形式的线性化 ····· 61
3.2.5 含 max/min 形式的线性化 ····· 62
**第 4 章 大规模线性规划的对偶** ····· 64
4.1 对偶理论概述 ····· 64
4.2 原问题与对偶问题之间的关系 ····· 65
4.3 对偶理论相关重要定理 ····· 66
4.4 最短路问题的对偶 ····· 73
4.4.1 借助 Excel 和具体小算例写出大规模 SPP 的对偶 ····· 73
4.4.2 SPP 中存在负环的特例 ····· 76
4.5 多商品网络流问题的对偶 ····· 81
4.5.1 借助 Excel 和具体小算例写出大规模 MCNF 的对偶 ····· 81
4.5.2 将 Excel 中的对偶问题表转化成公式形式 ····· 83
4.5.3 Python 调用 Gurobi 求解 MCNF ····· 83

## 第 II 部分 常用优化求解器 API 详解及应用案例


第 5 章 CPLEX 的 Java API 详解及简单案例 ····· 93
5.1 基于 Eclipse 的 CPLEX Java API 的安装与配置：适用于 macOS ····· 93
5.2 基于 Eclipse 的 CPLEX Java API 的安装与配置：适用于 Windows ····· 97
5.2.1 基本环境配置 ····· 97
5.2.2 环境设置中 java.library.path 的问题 ····· 98
5.2.3 Javadoc 的设置 ····· 100
5.3 CPLEX 建模 ····· 102
5.3.1 类与接口 ····· 102
5.3.2 变量 ····· 102
5.3.3 表达式 ····· 102
5.3.4 范围约束 ····· 104
5.3.5 目标函数 ····· 104
5.3.6 建模方式 ····· 105
5.3.7 模型求解 ····· 108
5.3.8 获取解的信息 ····· 108
5.3.9 模型导出与模型导入 ····· 109
5.3.10 参数 ····· 109
5.3.11 其他 ····· 114
5.4 传统回调 ····· 114
5.4.1 参考回调 ····· 115
5.4.2 查询或诊断回调 ····· 116
5.4.3 控制回调 ····· 117
5.4.4 回调的终止 ····· 118
5.4.5 传统回调示例 ····· 119
5.5 通用回调 ····· 121
5.5.1 简介 ····· 121
5.5.2 功能 ····· 121
5.5.3 通用回调的上下文 ····· 121
5.5.4 通用回调示例 ····· 123
5.6 Java 调用 CPLEX 求解整数规划的小例子 ····· 124
5.6.1 书架生产问题 ····· 124
5.6.2 包装盒选择问题 ····· 128
第 6 章 Gurobi 的 Python API 详解及简单案例 ····· 132
6.1 Python 调用 Gurobi 环境配置 ····· 132
6.1.1 完整步骤 ····· 132

6.1.2 相关问题 132
6.2 Gurobi 算法框架介绍 138
6.2.1 Gurobi MIP Algorithms 138
6.2.2 Presolving 140
6.2.3 Node Selection 141
6.2.4 Cutting Planes 142
6.2.5 Heuristics 143
6.2.6 设置启发式的参数 144
6.2.7 Branching 144
6.3 Gurobi 能够求解的模型类别 145
6.3.1 线性规划 145
6.3.2 混合整数规划 147
6.3.3 二次规划 148
6.3.4 二次约束二次规划 150
6.3.5 二阶锥规划 152
6.4 Python 调用 Gurobi 总体流程 154
6.5 Gurobi 求解 MIP 输出的日志信息解释 156
6.5.1 MIP 日志示例 156
6.5.2 预求解部分 157
6.5.3 求解进程部分 158
6.5.4 汇总部分 160
6.6 Python 接口概述 161
6.6.1 模型概述 161
6.6.2 求解模型 162
6.6.3 多个解、目标函数和场景 162
6.6.4 不可行的模型 162
6.6.5 查询和修改模型属性 163
6.6.6 其他修改模型信息的方法 163
6.6.7 惰性更新 164
6.6.8 参数管理 164
6.6.9 管理求解进程：日志和回调 165
6.6.10 修改求解器的行为：回调 165
6.7 Python 调用 Gurobi 常用类和函数 165
6.7.1 全局函数 165
6.7.2 Model 类 166
6.7.3 Var 类和 MVar 类 169

6.7.4 Column 类 ····· 170
6.7.5 目标函数 ····· 170
6.7.6 表达式 ····· 171
6.7.7 约束类 ····· 172
6.7.8 求解 ····· 175
6.7.9 解的输出 ····· 176
6.8 Python 接口中的 GRB 类 ····· 176
6.8.1 GRB 类中的常量 ····· 176
6.8.2 GRB 类中的属性：GRB.Attr ····· 180
6.8.3 GRB 类中的参数：GRB.Param ····· 185
6.9 Gurobi 的回调函数 ····· 190
6.9.1 什么是 Gurobi 的回调函数 ····· 190
6.9.2 Gurobi 回调函数的用法 ····· 192
6.10 Python 调用 Gurobi 的参数调优 ····· 193
6.11 Python 调用 Gurobi 求解整数规划的简单例子 ····· 194
**第 7 章 调用 CPLEX 和 Gurobi 求解 MIP 的复杂案例：VRPTW 和 TSP** ··· 197
7.1 调用 CPLEX 和 Gurobi 求解 VRPTW ····· 197
7.1.1 VRPTW 的一般模型 ····· 197
7.1.2 Java 调用 CPLEX 求解 VRPTW ····· 198
7.1.3 Java 调用 Gurobi 求解 VRPTW ····· 213
7.1.4 Python 调用 Gurobi 求解 VRPTW ····· 221
7.2 Python 调用 Gurobi 求解 TSP ····· 232
7.2.1 TSP 的 MTZ 建模及调用 Gurobi 求解 ····· 235
7.2.2 TSP：Python 调用 Gurobi 实现 callback 添加消除子环路约束 ····· 237

## 第 III 部分 运筹优化常用算法及实战

**第 8 章 单纯形法** ····· 249
8.1 线性规划问题的标准形式 ····· 249
8.2 单纯形法流程图及详细案例 ····· 251
8.3 大 $M$ 法和两阶段法 ····· 256
8.4 单纯形法伪代码 ····· 257
8.5 Python 实现单纯形法 ····· 295
**第 9 章 Dijkstra 算法** ····· 264
9.1 Dijkstra 算法求解最短路问题详解 ····· 264
9.2 Dijkstra 算法步骤及伪代码 ····· 268

9.3 Python 实现 Dijkstra 算法 ····· 270
9.3.1 网络数据准备 ····· 270
9.3.2 Dijkstra 算法实现 ····· 271
9.3.3 算例测试 ····· 272
9.4 拓展 ····· 273
第 10 章 分支定界算法 ····· 274
10.1 整数规划和混合整数规划 ····· 274
10.2 分支定界算法求解混合整数规划 ····· 275
10.3 分支定界算法的一般步骤和伪代码 ····· 285
10.4 分支定界算法执行过程的直观展示 ····· 289
10.5 分支定界算法的分支策略 ····· 290
10.6 分支定界算法的搜索策略 ····· 291
10.7 分支定界算法的剪枝策略 ····· 291
10.8 Python 调用 Gurobi 实现分支定界算法的简单案例 ····· 292
10.9 Python 调用 Gurobi 实现分支定界算法求解 TSP ····· 299
10.10 Python 调用 Gurobi 实现分支定界算法求解 VRPTW ····· 300
第 11 章 分支切割算法 ····· 302
11.1 什么是分支切割算法 ····· 302
11.2 有效不等式 ····· 305
11.3 割平面算法 ····· 307
11.3.1 Gomory's 分数割平面算法 ····· 308
11.3.2 其他割平面算法 ····· 312
11.4 分支切割算法: 分支定界 + 割平面 ····· 313
11.4.1 分支切割算法伪代码 ····· 313
11.4.2 分支切割算法: 一个详细的例子 ····· 316
11.5 Java 调用 CPLEX 实现分支切割算法求解 VRPTW ····· 318
11.5.1 分支定界 ····· 318
11.5.2 割平面 ····· 319
11.6 Python 调用 Gurobi 实现分支切割算法求解 VRPTW 完整代码 ····· 320
11.7 Java 调用 CPLEX 实现分支切割算法求解 CVRP: 回调函数添加割平面 ····· 321
11.7.1 CVRP 的基本模型 ····· 321
11.7.2 割平面 ····· 322
第 12 章 拉格朗日松弛 ····· 325
12.1 最优性和松弛 ····· 325
12.1.1 原始边界 ····· 326
12.1.2 对偶边界 ····· 327

12.2 对偶……327
12.3 拉格朗日松弛……328
12.3.1 拉格朗日松弛介绍……328
12.3.2 拉格朗日对偶问题……329
12.3.3 拉格朗日松弛应用案例：无容量限制的设施选址问题……330
12.4 拉格朗日对偶的加强……332
12.5 求解拉格朗日对偶……334
12.5.1 次梯度算法求解拉格朗日对偶……334
12.5.2 应用案例：拉格朗日松弛求解 TSP……337
12.6 如何选择拉格朗日松弛……339
12.7 Python 调用 Gurobi 实现拉格朗日松弛求解选址–运输问题……341
12.7.1 拉格朗日松弛应用案例：选址–运输问题……341
12.7.2 Python 代码实现……344
**第 13 章 列生成算法**……346
13.1 为什么用列生成算法……346
13.2 下料问题……348
13.2.1 引例……348
13.2.2 列生成求解下料问题的一般模型及其伪代码……351
13.2.3 列生成最优性的几个小问题……354
13.3 列生成求解下料问题的实现……355
13.3.1 Python 调用 Gurobi 实现列生成求解下料问题示例算例：版本 1……356
13.3.2 Python 调用 Gurobi 实现列生成求解下料问题示例算例：版本 2（以人工变量为初始列的方式）……359
13.3.3 Python 调用 Gurobi 实现列生成求解下料问题：版本 3……361
13.4 列生成求解 TSP……364
13.4.1 TSP 的 1-tree 建模及列生成求解……364
13.4.2 主问题……365
13.4.3 子问题……366
13.5 列生成求解 VRPTW……368
13.5.1 主问题……369
13.5.2 子问题……371
13.5.3 详细案例演示……372
**第 14 章 动态规划**……392
14.1 动态规划……392
14.1.1 动态规划求解最短路问题……393
14.1.2 问题建模和求解……393

14.1.3 一个较大规模的例子 ····· 395
14.2 动态规划的实现 ····· 396
14.2.1 伪代码 ····· 396
14.2.2 Java 代码 ····· 397
14.3 动态规划求解 TSP ····· 402
14.3.1 一个简单的 TSP 算例 ····· 402
14.3.2 伪代码 ····· 407
14.3.3 Python 实现：示例算例 ····· 408
14.3.4 Python 实现：中大规模算例 ····· 411
14.4 标签算法求解带资源约束的最短路问题 ····· 418
14.4.1 带资源约束的最短路问题 ····· 418
14.4.2 标签算法 ····· 423
14.4.3 标签算法的伪代码 ····· 424
14.4.4 标签设定算法和标签校正算法 ····· 425
14.4.5 优超准则和优超算法 ····· 425
14.4.6 Python 实现标签算法求解 SPPRC ····· 426
14.5 Python 实现标签算法结合列生成求解 VRPTW ····· 434
14.5.1 初始化 RMP ····· 434
14.5.2 标签算法求解子问题 ····· 436
**第 15 章 分支定价算法** ····· 437
15.1 分支定价算法基本原理概述 ····· 437
15.2 分支定价算法求解 VRPTW ····· 439
15.2.1 VRPTW 的通用列生成建模方法 ····· 439
15.2.2 分支定价算法完整流程及伪代码 ····· 441
15.2.3 分支策略 ····· 444
15.2.4 界限更新 ····· 448
**第 16 章 Dantzig-Wolfe 分解算法** ····· 449
16.1 引例 ····· 449
16.2 块角模型与 Dantzig-Wolfe 分解 ····· 453
16.2.1 块角模型 ····· 453
16.2.2 Minkowski 表示定理 ····· 454
16.2.3 模型分解 ····· 454
16.2.4 两阶段法 ····· 457
16.3 详细案例 ····· 458
16.4 Dantzig-Wolfe 分解求解大规模混合整数规划 ····· 463
16.4.1 两阶段法实现 Dantzig-Wolfe 分解算法介绍 ····· 464

16.4.2 第 1 阶段 ····· 464
16.4.3 第 2 阶段 ····· 470
16.5 Python 调用 Gurobi 实现 Dantzig-Wolfe 分解求解多商品流运输问题 ····· 481
16.5.1 多商品网络流模型的区块结构 ····· 481
16.5.2 多商品流运输问题: Dantzig-Wolfe 分解求解 ····· 482
16.5.3 Python 调用 Gurobi 实现 Dantzig-Wolfe 分解求解多商品流运输问题 ····· 486
16.5.4 完整代码 ····· 486
16.5.5 算例格式说明 ····· 496
16.5.6 算例运行结果 ····· 497
16.6 Dantzig-Wolfe 分解求解 VRPTW ····· 499
**第 17 章 Benders 分解算法** ····· 503
17.1 分解方法 ····· 503
17.1.1 Benders 分解的原理 ····· 503
17.1.2 Benders 分解的全过程 ····· 508
17.1.3 算法框架图 ····· 509
17.1.4 算法伪代码 ····· 510
17.2 详细案例 ····· 511
17.2.1 问题描述和模型转换 ····· 511
17.2.2 第 1 次迭代 ····· 512
17.2.3 第 2 次迭代 ····· 514
17.2.4 第 3 次迭代 ····· 515
17.2.5 第 4 次迭代 ····· 516
17.3 Benders 分解应用案例 ····· 517
17.3.1 固定费用运输问题 ····· 517
17.3.2 设施选址问题 ····· 519
参考文献 ····· 522

# PART ONE

*第I部分*

## 运筹优化常用模型及建模技巧

# 第1章　运筹优化算法相关概念

本章主要介绍一些运筹优化相关的概念，包括常见的数学规划模型以及凸集、极点、多面体、超平面与半空间等。本章旨在为后续章节做铺垫，因此仅对涉及的概念做简要介绍，需要深入了解的读者请参考文献（Boyd et al.，2004）。

## 1.1　几类常见的数学规划模型

本书探讨的优化问题均是约束最优化问题，其通式为

$$\min\ (\text{or}\max) \quad f(x) \tag{1.1}$$

$$\text{s.t.} \quad \begin{cases} g(x) \leqslant 0 & (1.2) \\ x \in X & (1.3) \end{cases}$$

其中，式 (1.1) 为目标函数，可以为线性或者非线性表达式。式 (1.2) 和式 (1.3) 共同构成该优化问题的约束条件。其中约束 (1.2) 也可以为线性或者非线性表达式。式（1.3）为决策变量的取值范围和类型的约束，$X$ 为决策变量 $x$ 的取值要求（范围和类型）；如果 $x$ 是连续变量，且 $g(x)$ 为线性表达式，则 $X$ 是一个多面体；如果 $x$ 是整数变量（离散变量），则 $X$ 是一系列离散的点集。

根据 $f(x), g(x), X$ 的不同类型，可以将上述约束优化问题分成多种不同种类的优化问题模型，其中，比较常见的有以下几种。

### 1.1.1　线性规划

线性规划（Linear Programming, LP）是指目标函数和约束条件均为线性的数学规划，它是数学规划的基础，同时也是关键。线性规划的标准型形式如下：

$$\begin{aligned} \min \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}, \quad \boldsymbol{x} \in \mathbb{R}^{n\times 1} \end{aligned} \tag{LP}$$

其中，$\boldsymbol{c} \in \mathbb{R}^{n\times 1}$，是 $n$ 维的行向量，$\boldsymbol{A} \in \mathbb{R}^{m\times n}$，$\boldsymbol{b} \in \mathbb{R}^{m\times 1}$，$\boldsymbol{c}, \boldsymbol{A}, \boldsymbol{b}$ 均为已知参数，$\boldsymbol{x}$ 是 $n \times 1$ 维的列向量。

### 1.1.2　混合整数规划

当线性规划的所有决策变量都要求取值为整数时，该线性规划就变成了一个整数规划（Integer Programming, IP），或者纯整数规划（Pure Integer Programming）。但是当只

有一部分决策变量要求取值为整数时，该线性规划就变为混合整数规划（Mixed Integer Programming, MIP）。混合整数规划也是本书重点讨论的内容，其一般形式为

$$\begin{aligned} \min \quad & \boldsymbol{m}^{\mathrm{T}}\boldsymbol{x}+\boldsymbol{n}^{\mathrm{T}}\boldsymbol{y} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x}+\boldsymbol{C}\boldsymbol{y}\leqslant\boldsymbol{b} \\ & \boldsymbol{x}\in\mathbb{R}^n, \boldsymbol{y}\in\mathbb{Z}^n \end{aligned} \tag{MIP}$$

### 1.1.3 二次规划

二次规划（Quadratic Programming, QP）是指目标函数含有二次项、约束为线性约束的数学规划，其一般形式为

$$\begin{aligned} \min \quad & \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{x}+\boldsymbol{q}^{\mathrm{T}}\boldsymbol{x}+r \\ \text{s.t.} \quad & \begin{cases} \boldsymbol{G}\boldsymbol{x}\leqslant\boldsymbol{h} \\ \boldsymbol{A}\boldsymbol{x}=\boldsymbol{b} \end{cases} \end{aligned} \tag{QP}$$

其中，目标函数 $\frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{x}+\boldsymbol{q}^{\mathrm{T}}\boldsymbol{x}+r$ 为二次表达式，但是约束为线性约束。如果要求一部分决策变量为整数，则 QP 变为混合整数二次规划（Mixed Integer Quadratic Programming, MIQP）。

### 1.1.4 二次约束规划

二次约束规划（Quadratically Constrained Programming, QCP）是指目标函数为线性表达式、约束含有二次项的数学规划，其一般形式为

$$\begin{aligned} \min \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.} \quad & \begin{cases} \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}_i\boldsymbol{x}+\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{x}+r_i\leqslant 0, & \forall i=1,2,\cdots,m \\ \boldsymbol{A}\boldsymbol{x}=\boldsymbol{b} \end{cases} \end{aligned} \tag{QCP}$$

其中，第一个约束的左端项 $\frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{x}+\boldsymbol{q}^{\mathrm{T}}\boldsymbol{x}+r$ 为二次表达式，目标函数 $\boldsymbol{c}\boldsymbol{x}$ 为线性表达式。如果要求一部分决策变量为整数，则 QCP 变为混合整数二次约束规划（Mixed Integer Quadratically Constrained Programming, MIQCP）。

### 1.1.5 二次约束二次规划

二次约束二次规划（Quadratically Constrained Quadratic Programming, QCQP）是指目标函数为二次表达式、约束也为二次约束的数学规划。

当 QCP 中的目标函数也是二次表达式时，QCP 就变成了 QCQP，其一般形式为

$$\min \quad \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}_0\boldsymbol{x}+\boldsymbol{q}_0^{\mathrm{T}}\boldsymbol{x}+r_0 \tag{QCQP}$$

$$\text{s.t.} \quad \begin{cases} \dfrac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}_i\boldsymbol{x}+\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{x}+r_i \leqslant 0, & \forall i=1,2,\cdots,m \\ \boldsymbol{A}\boldsymbol{x}=\boldsymbol{b} \end{cases}$$

如果要求一部分决策变量为整数，则 QCQP 变为混合整数二次约束二次规划（Mixed Integer Quadratically Constrained Quadratic Programming, MIQCQP）。

### 1.1.6 二阶锥规划

二阶锥规划（Second-Order Cone Programming, SOCP）是指约束中含有二阶锥约束、但是目标函数为线性表达式的数学规划，其一般形式为

$$\begin{aligned} \min \quad & \boldsymbol{f}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.} \quad & \|\boldsymbol{A}_i\boldsymbol{x}+\boldsymbol{b}_i\|_2 \leqslant \boldsymbol{c}_i^{\mathrm{T}}\boldsymbol{x}+d_i, \qquad \forall i=1,2,\cdots,m \\ & \boldsymbol{F}\boldsymbol{x}=\boldsymbol{g} \end{aligned} \tag{SOCP}$$

其中，$\boldsymbol{x}\in\mathbb{R}^n$, $\boldsymbol{A}_i\in\mathbb{R}^{n_i\times n}$, $\boldsymbol{b}_i\in\mathbb{R}^{n_i}$, $\boldsymbol{c}_i\in\mathbb{R}^n$, $d_i\in\mathbb{R}$, $\boldsymbol{F}\in\mathbb{R}^{p\times n}$, $\boldsymbol{g}\in\mathbb{R}^p$。形如

$$\|\boldsymbol{A}\boldsymbol{x}+\boldsymbol{b}\|_2 \leqslant \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}+d$$

的约束称为二阶锥约束（Second-Order Cone Constraint）。例如：

$$\sqrt{x_1^2+x_2^2} \leqslant 4y+6$$

SOCP 可以使用内点法高效求解，很多求解器也可以直接求解 SOCP（如 Gurobi、CPLEX 等）。现实中，大量问题都能转化为 SOCP 来求解。如果 SOCP 中要求一部分决策变量为整数，则 SOCP 变为混合整数二阶锥规划（Mixed Integer Second-Order Cone Programming, MISOCP）。

SOCP 和 LP、QCP 是有一定关系的。当 $\boldsymbol{A}_i=0,\forall i=1,2,\cdots,m$ 时，SOCP 退化为 LP。当 $\boldsymbol{c}_i=0,\forall i=1,2,\cdots,m$ 时，SOCP 等价于 QCP，因为此时，二阶锥约束可以通过平方转化成二次约束。比如约束

$$\sqrt{x_1^2+x_2^2} \leqslant 6$$

可以通过两边同时平方转化成

$$x_1^2+x_2^2 \leqslant 36$$

下面我们用一个示意图形象地理解二阶锥约束。设三维空间 $\mathbb{R}^3$ 中的二阶锥（基向量为 $(x_1,x_2,y)$）

$$y \geqslant \sqrt{x_1^2+x_2^2}$$

被平面 $y=1$ 切割以后的部分如图 1.1 所示。可以看到，其形状就是一个锥。由于约束表达式采用的是二范数，所以叫作二阶锥。

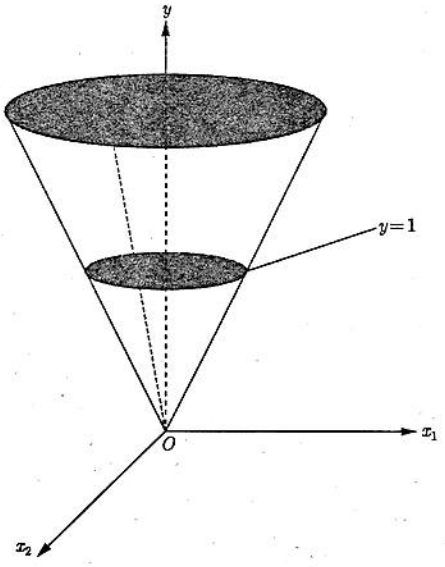



图 1.1 二阶锥 $y \geqslant \sqrt{x_1^2 + x_2^2}$ 被平面 $y = 1$ 切割

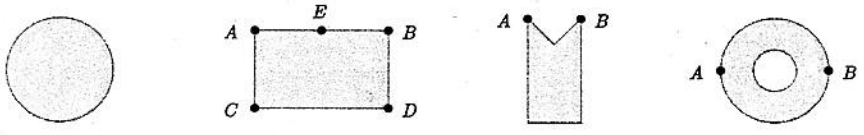
当然，除了上述几种数学规划模型，还有半正定规划（Semi-Definite Programming, SDP）等, 在此我们不做详细介绍，感兴趣的读者可以参阅文献（Boyd et al., 2004）。

## 1.2 凸集和极点

### 1.2.1 凸集

当连接点集 $S$ 中任意两点的线段完全包含在 $S$ 中时，则称 $S$ 为凸集（Convex Set）（Boyd et al., 2004）。可用数学表达式表示为：对于集合 $S$，取任意两个点 $x, y \in S$，如果对于任意的 $\lambda \in [0,1]$，有 $\lambda x + (1-\lambda)y \in S$，那么集合 $S$ 是凸集。

我们通过如下四个图来进一步理解凸集的含义。对于图 1.2(a) 和图 1.2(b) 来说，连接集合 $S$ 中任意两个点的每条线段，都只包含 $S$ 中的点，因此在这两个图中，$S$ 是凸集。而对于图 1.2(c) 和图 1.2(d) 来说，$S$ 不是凸集。在这两个图中，点 $A$ 和点 $B$ 在集合 $S$ 中，但是线段 $AB$ 上的某些点不在集合 $S$ 中。




(a) 集合$S$ (b) 集合$S$ (c) 集合$S$ (d) 集合$S$

图 1.2 凸集和非凸集

### 1.2.2 极点

对于任意凸集 $S$，当完全在 $S$ 中并且包含点 $P$ 的每条线段都以 $P$ 作为线段的端点时，则称 $S$ 中的点 $P$ 为极点（Extreme Point）。

我们通过图 1.2(a) 和图 1.2(b) 来理解极点含义。对于图 1.2(a) 来说，实心圆周上的每个点都是这个实心圆的极点。对于图 1.2(b) 来说，点 $A$、$B$、$C$ 和 $D$ 是 $S$ 的极点。虽然点 $E$ 在 $S$ 的边界上，但是因为点 $E$ 在线段 $AB$ 上（$AB$ 完全在 $S$ 中）而点 $E$ 却不是线段 $AB$ 的端点，所以点 $E$ 不是 $S$ 的极点。

极点有时也称作角点（Corner Point），因为当集合 $S$ 是一个多边形时，$S$ 的极点是这个多边形的顶点或者角点。

## 1.3 多面体、超平面与半空间

### 1.3.1 多面体

多面体（Polyhedron）是一个集合，其形式可以描述为 $\{\boldsymbol{x} \in \mathbb{R}^n | \boldsymbol{A}\boldsymbol{x} \geqslant \boldsymbol{b}\}$，$\boldsymbol{A}$ 是 $m \times n$ 的矩阵，$\boldsymbol{b}$ 是 $m$ 维列向量（Boyd et al., 2004）。多面体是有限个超平面和半空间的交集，如图 1.3(a) 和图 1.3(b) 所示。

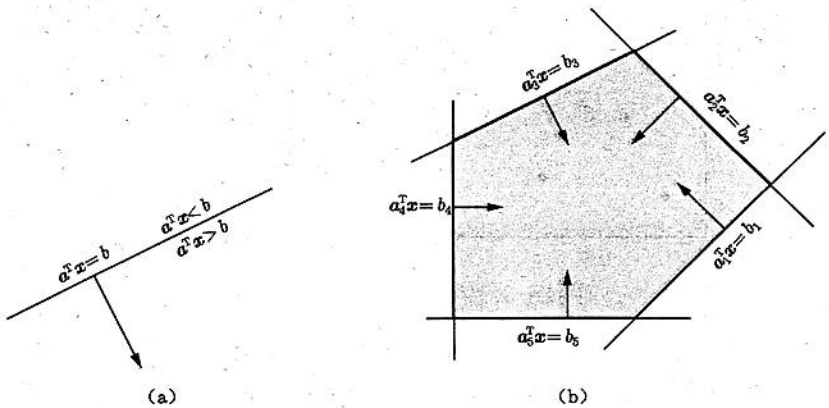



图 1.3 多面体、超平面与半平面

对于线性规划而言，其可行域可以描述成不等式约束 $\boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}$ 的形式，因此也是一个多面体。

### 1.3.2 超平面与半空间

设 $\boldsymbol{a}$ 是一个 $\mathbb{R}^n$ 非零向量，$b$ 是一个标量，则：（1）集合 $\{\boldsymbol{x} \in \mathbb{R}^n | \boldsymbol{a}_1^{\mathrm{T}}\boldsymbol{x} = b\}$ 称作超平面（Hyperplane, Boyd et al., 2004）；（2）集合 $\{\boldsymbol{x} \in \mathbb{R}^n | \boldsymbol{a}_2^{\mathrm{T}}\boldsymbol{x} \geqslant b\}$ 称作半空间（Halfspace, Boyd et al., 2004）。

可以发现，超平面是相应半空间的边界。此外，向量 $a$ 在超平面的定义中是垂直于自身对应的半空间的。我们可以通过图 1.3(a) 和图 1.3(b) 得到一些直观的理解。

如图 1.3 所示，图 1.3(a) 展示了一个超平面与两个半空间，图 1.3(b) 展示了由五个半空间围起来的多面体 $\{x|a_i^{\mathrm{T}}x \geqslant b_i, i=1,2,\cdots,5\}$。同时每一个向量 $a_i$ 都垂直于相应的超平面 $\{x|a_i^{\mathrm{T}}x = b_i\}$。

## 1.4 凸组合和凸包

### 1.4.1 凸组合和凸包的概念

设 $x^1, x^2, \cdots, x^k$ 是 $k$ 个 $n$ 维向量，即 $x^i \in \mathbb{R}^n, \forall i = 1,2,\cdots,k$。$\lambda_1, \lambda_2, \cdots, \lambda_k$ 是 $k$ 个非负标量，满足 $\sum_{i=1}^{k} \lambda_i = 1$，则：(1) 向量 $\sum_{i=1}^{k} \lambda_i x^i$ 是向量 $x^1, \cdots, x^k$ 的凸组合（Convex Combination）(Boyd et al., 2004)；(2) 向量 $x^1, x^2, \cdots, x^k$ 的凸包（Convex Hull）是这些向量的所有凸组合组成的集合 (Boyd et al., 2004)。

凸包是整数规划相关的重要概念，我们通过图 1.4 来直观认识一下凸包。

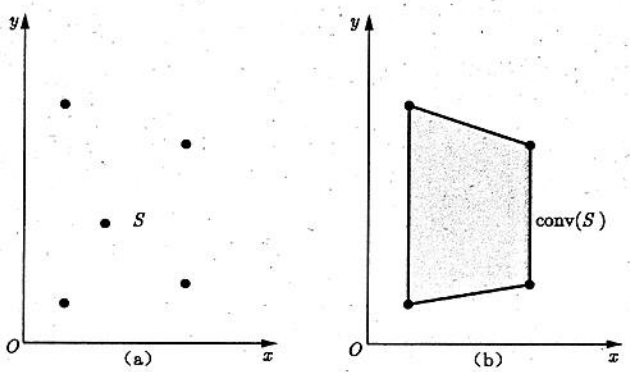



图 1.4 凸包在二维空间上的示意图

图 1.4 展示了凸包在二维空间上的示意图。图 1.4(a) 展示了集合 $S$ 是由 5 个点组成的集合，那么集合 $S$ 的凸包就是包含集合 $S$ 的最小的凸集，如图 1.4(b) 的阴影区域所示。我们将集合 $S$ 的凸包记作 $\mathrm{conv}(S)$。

### 1.4.2 一些结论

下面我们不加证明地给出下面的结论。

(1) 凸集的交集仍是凸集。

(2) 每一个多面体都是一个凸集。

(3) 一个凸集中的有限个元素组成的凸组合仍然属于该集合。

(4) 由有限向量组成的凸包是一个凸集。

详细证明参见文献 (Boyd et al., 2004)。

# 第2章　运筹优化经典问题数学模型

## 2.1　指派问题

指派问题（Assignment Problem）是一类非常常见的运筹学问题。例如，某公司有 $N$ 项必须完成的任务和 $N$ 个可以完成这些任务的员工，假设每个员工都可以胜任所有的 $N$ 项任务，只是效率有所不同。为了以最快的效率完成这 $N$ 项任务，公司需要为每一位员工分配一项任务，并且每一项任务只能由一个员工承担。这就是典型的指派问题，这类问题在实际中应用非常广泛。

以时下非常受关注的出行服务为例（代表企业有 Uber、滴滴、Lyft 等）。假设在某一时间段内，出行服务平台的派单系统内可被分配的司机和订单数目均为 $N$，为了区分，假设司机的集合为 $D$，乘客的集合为 $R$，且 $|D| = |R| = N$。出行服务平台需要为每一位乘客 $i \in R$ 分配一个司机 $j \in D$，每一个分配都会对应一个成本 $c_{ij}$（可以认为是这个订单的距离），而出行服务平台的目标是找到总距离最小的指派策略（实际中，还有可能是最大化平台的收益等）。

为了解决这个问题，可以引入一组 0-1 决策变量 $x_{ij}$，含义为

$$x_{ij} = \begin{cases} 1, & \text{如果乘客 } i \text{ 被分配给司机 } j \\ 0, & \text{其他} \end{cases}$$

那么，该指派问题可以被建模为下面的整数规划模型：

$$\min \sum_{i \in R} \sum_{j \in D} x_{ij} c_{ij} \tag{2.1}$$

$$\text{s.t.} \sum_{j \in D} x_{ij} = 1, \quad \forall i \in R \tag{2.2}$$

$$\sum_{i \in R} x_{ij} = 1, \quad \forall j \in D \tag{2.3}$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in R, j \in D \tag{2.4}$$

该问题是一种特殊的二部图匹配（Bipartite Matching）问题，可以表示为图 2.1 的形式。其中，$D_i$ $(i = 1, 2, 3, 4)$ 表示司机，$R_i$ $(i = 1, 2, 3, 4)$ 表示乘客。虚线连接的司机和乘客表示我们可以将该司机和乘客进行匹配，实线表示最优指派方案。具体应用案例参见文献（Xu et al., 2018）。

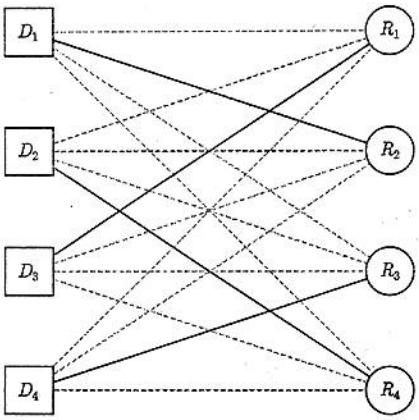



图 2.1 指派问题（二部图匹配）

虽然该问题可以通过建立整数规划模型，然后调用优化求解器（CPLEX、Gurobi 等）快速求解，但是该算法也存在更为高效的多项式时间精确解法，包括匈牙利算法（Hungarian Algorithm，也叫 Kuhn-Munkres Algorithm，(Kuhn 1955; Munkres 1957)）、Ford-Fulkerson 算法（也叫 Edmonds-Karp Algorithm，(Edmonds and Karp 1972)）、Hopcroft-Karp 算法（也叫 Hopcroft-Karp-Karzanov Algorithm，(Hopcroft and Karp 1973)）。这些算法都是由赫赫有名的科学家提出，感兴趣的读者可以去详细了解。匈牙利算法的原始版本的复杂度为 $O(n^4)$，后来有研究者将该算法进行了提升，其复杂度降低到 $O(n^3)$。Ford-Fulkerson 算法和 Hopcroft-Karp 算法的算法复杂度分别为 $O(Ef)$ 和 $O(E\sqrt{V})$，其中 $E$ 是二部图中边（或者弧）的个数，$f$ 是最大流的值，$V$ 是二部图中节点的个数。Ford-Fulkerson 算法和 Hopcroft-Karp 算法都是基于将二部图匹配转化成最大流问题（见图 2.2）的思想而提出的算法。本节对此不再做详细展开。

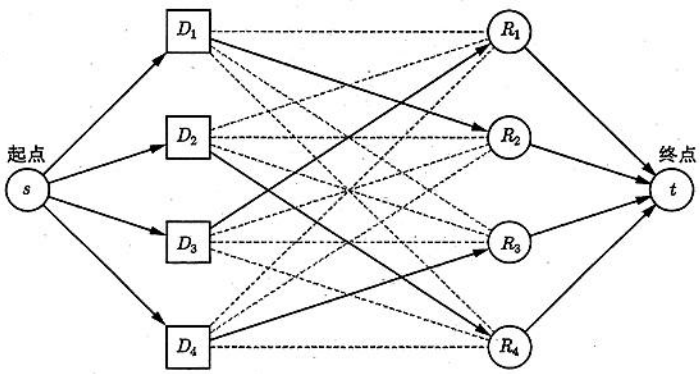



图 2.2 二部图匹配问题转化为最大流问题

## 2.2 最短路问题

最短路问题（Shortest Path Problem, SPP）是一类非常经典的问题。最短路问题可以描述为：给定一个有向图（或无向图）$G=(V,E)$，$V$ 是图中点的集合，$E$ 是图中边的集合。图中的每条边 $(i,j)\in E$ 都对应一个权重 $c_{ij}$。给定一个起点 $s(s\in V)$ 和一个终点 $t(t\in V)$，最短路问题就是去找到一条从 $s$ 出发，到达 $t$ 的距离或者成本最小的路径。最基本的最短路问题并不是 NP-hard 问题，可以用 Dijkstra 等算法在多项式时间内求解到最优解（Dijkstra et al., 1959），Dijkstra 算法的复杂度为 $\mathcal{O}(|V|^2)$，并且在不同数据结构下，复杂度略有不同。

我们首先来看一个示例网络，如图 2.3 所示（该例子来自文献 Cappanera and Scaparra 2011），图中每条边上的数字代表该条边的权重（或者成本、行驶距离等）。图中最短路径为 $1\to 2\to 4\to 3\to 6\to 7$，总长度为 45。

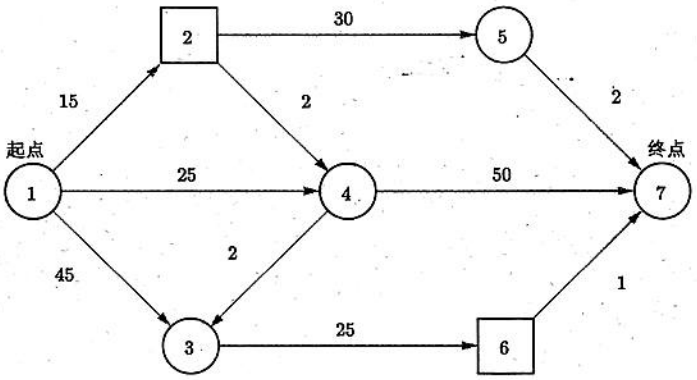



图 2.3 最短路问题：示例网络

最短路问题可以建模为一个整数规划问题。我们首先引入 0-1 决策变量 $x_{ij}$ 如下：

$$x_{ij}=\begin{cases}1, & \text{如果边 } (i,j) \text{ 在最优解中被选中}\\ 0, & \text{其他}\end{cases}$$

然后，最短路问题的数学模型可以表示为

$$\min \quad \sum_{e\in E} d_e x_e \tag{2.5}$$

$$\text{s.t.} \quad \sum_{e\in \text{out}(i)} x_e - \sum_{e\in \text{in}(i)} x_e = \begin{cases}1, & i=s\\ -1, & i=t\\ 0, & \text{其他}\end{cases} \tag{2.6}$$

$$x_e\in\{0,1\}, \qquad \forall e\in E \tag{2.7}$$

其中，$\text{out}(i)$ 表示离开点 $i$ 的边的集合，$\text{in}(i)$ 表示进入点 $i$ 的边的集合。上述模型中，约束 (2.6) 还可以写为

$$\sum_{e\in\text{out}(i)} x_e - \sum_{e\in\text{in}(i)} x_e = b_i, \qquad \forall i \in V \tag{2.8}$$

其中，当 $i=s$ 时，$b_i=1$；当 $i=t$ 时，$b_i=-1$；否则 $b_i=0$。两种写法都比较常见。

该问题具有整数最优解特性，也就是说，即使把变量 $x_{ij}$ 松弛成 $0\leqslant x_{ij}\leqslant 1$，原问题变成线性规划，也同样存在整数最优解，关于该性质更为详细的介绍，见第 2.4 节。另外，在一些高水平期刊中，也不乏该性质的身影，参见文献（Cappanera and Scaparra, 2011）。根据上述介绍，最短路问题的整数规划模型等价于下面的线性规划：

$$\min \quad \sum_{e\in E} d_e x_e \tag{2.9}$$

$$\text{s.t.} \quad \sum_{e\in\text{out}(i)} x_e - \sum_{e\in\text{in}(i)} x_e = b_i, \qquad \forall i \in V \tag{2.10}$$

$$0\leqslant x_e \leqslant 1, \qquad \forall e \in E \tag{2.11}$$

关于最短路问题更为深入的探讨，参见第 4 章和第 9 章。

## 2.3 最大流问题

很多生产应用场景都可以将模型构建为一个弧段有流量限制的网络问题，在这个网络中，通常需要将最大量的商品从一个源点运送到一个汇点，这类问题就是经典的最大流问题（Maximum Flow Problem）。

### 2.3.1 问题描述

下面用一个例子解释如何将最大流问题转化为线性规划问题（Winston and Goldberg, 2004）。

某油气加工公司要进行油料输送，输送网络是具有源点 (so) 和汇点 (si) 的管道网络，如图 2.4 所示。油料从源点到汇点要途经站点 1、2、3。不同站点之间的管道弧段具有不同的管道直径，不同管道的最大输送容量如表 2.1 所示，表中每个数表示弧段容量（百万桶/时）。构建线性规划求解该网络的最大输送能力，比如每小时输送几百万桶。

表 2.1 石油运输问题的弧容量

| 弧 | 容 量 |
|---|---|
| (so, 1) | 2 |
| (so, 2) | 3 |
| (1, 2) | 3 |
| (1, 3) | 4 |
| (3, si) | 1 |
| (2, si) | 2 |

将每个弧段的输送容量标注到图 2.4 中。

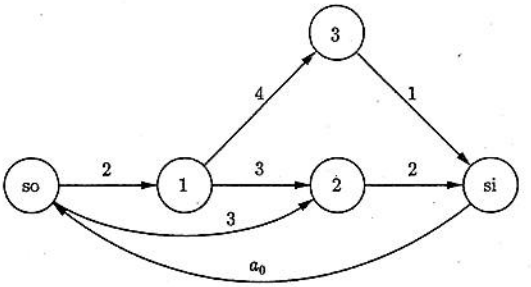



图 2.4 石油运输网络图

### 2.3.2 问题建模及最优解

为了方便建模，我们人工添加一个从汇点到源点的弧段，其容量设定为 $a_0$，这个弧段实际不进行油料输送，只是方便建模时保证流量平衡。我们设定 $x_{ij}$ 为决策变量，表示在弧段 $(i,j)$ 上每小时通过的油料量（百万桶）。其中一个可行的方案是如下配置：$x_{\mathrm{so},1}=2, x_{13}=0, x_{12}=2, x_{3,\mathrm{si}}=0, x_{2,\mathrm{si}}=2, x_{\mathrm{si,so}}=2, x_{\mathrm{so},2}=0$。该方案展示如图 2.5 所示，括号中的数字表示 $x_{ij}$ 的值，此时网络的最大流量为 2 百万桶/时。

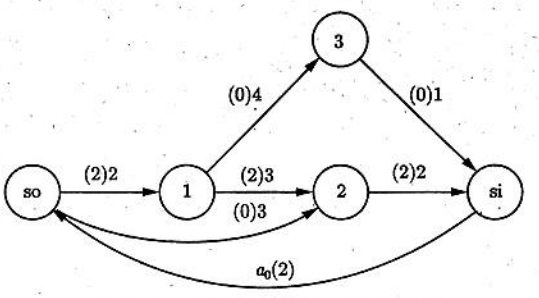



图 2.5 石油运输问题的一个可行解

容易分析得到，对于一个可行的流量设定，它必须满足容量约束

$$0 \leqslant x_{ij} \leqslant \text{弧 } (i,j) \text{ 的容量}, \quad \forall (i,j) \tag{2.12}$$

和流平衡约束

$$\text{流入点 } i \text{ 的流量} = \text{流出点 } i \text{ 的流量}, \quad \forall i \tag{2.13}$$

人工弧 (si, so) 和人工变量 $a_0$ 的添加就是为了保证源点和汇点的流量平衡。

下面构建最大流问题的线性规划模型：设 $x_0$ 表示从人工弧段 (si, so) 上流通的流量，根据流平衡约束可得 $x_0$ 等于所有进入汇点 si 的流量，也就是网络中流通的最大流量。油

气加工公司目标是最大化网络流量，也就是最大化 $x_0$，其模型可以表示如下：

$$\max \quad z = x_0 \tag{2.14}$$

$$\text{s.t.} \quad x_{\text{so},1} \leqslant 2 \tag{2.15}$$

$$x_{\text{so},2} \leqslant 3 \tag{2.16}$$

$$x_{12} \leqslant 3 \tag{2.17}$$

$$x_{2,\text{si}} \leqslant 2 \tag{2.18}$$

$$x_{13} \leqslant 4 \tag{2.19}$$

$$x_{3,\text{si}} \leqslant 1 \tag{2.20}$$

$$x_0 = x_{\text{so},1} + x_{\text{so},2} \tag{2.21}$$

$$x_{\text{so},1} = x_{12} + x_{13} \tag{2.22}$$

$$x_{\text{so},2} + x_{12} = x_{2,\text{si}} \tag{2.23}$$

$$x_{13} = x_{3,\text{si}} \tag{2.24}$$

$$x_{3,\text{si}} + x_{2,\text{si}} = x_0 \tag{2.25}$$

$$x_{ij} \geqslant 0, \quad \forall i,j \in \{\text{so},1,2,3,\text{si}\} \tag{2.26}$$

其中，约束 (2.15)~(2.20) 是弧容量约束，约束 (2.21)~(2.25) 是流平衡约束，最后的约束 (2.26) 是变量非负的物理含义约束。模型建好之后，可以考虑用求解器进行求解，本书有专门介绍求解器求解线性规划和整数规划的章节，这里不再赘述，读者请参见相关章节。模型求解后的最优解为 $z=3, x_{\text{so},1}=2, x_{13}=1, x_{12}=1, x_{3,\text{si}}=1, x_{2,\text{si}}=2, x_{\text{so},2}=1, x_0=x_{\text{si,so}}=3$。因此，网络中的最大流为 3 百万桶/时，分别以 1 百万桶/时的流量经过线路：$\text{so}\to 1\to 2\to \text{si}$、$\text{so}\to 1\to 3\to \text{si}$ 和 $\text{so}\to 2\to \text{si}$。最优流量分配情况如图 2.6 所示。

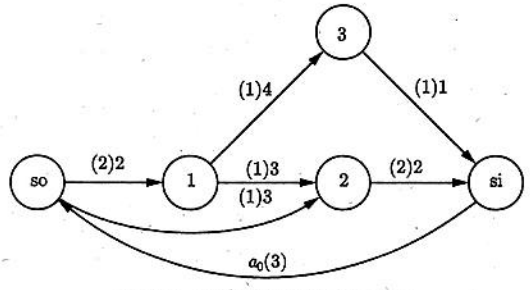



图 2.6　石油运输问题的最优解

### 2.3.3　最大流问题的一般模型

在前面的章节中，用一个非常小的例子介绍了最大流问题数学模型，这里给出最大流问题的一般模型。给定一个有向图 $G=(V,E)$，其中 $V$ 是图中点的集合，$E$ 是图中边的

集合。图中的每条边 $(i,j) \in E$ 都对应一个最大容量 $u_{ij}$。点 so 和 si 分别代表起点和终点。我们引入如下两个决策变量：

（1）$x_e$：边 $e \in E$ 上的流量；

（2）$f$：网络的最大流量；

则最大流问题的一般模型可以写为

$$\max \quad f \tag{2.27}$$

$$\text{s.t.} \quad \sum_{e \in \text{out}(i)} x_e - \sum_{e \in \text{in}(i)} x_e = b_i, \qquad \forall i \in V \tag{2.28}$$

$$0 \leqslant x_e \leqslant u_{ij}, \qquad \forall e \in E \tag{2.29}$$

其中，当 $i = \text{so}$ 时，$b_i = f$；当 $i = \text{si}$ 时，$b_i = -f$；否则 $b_i = 0$。

### 2.3.4 Ford-Fulkerson 算法求解最大流问题

本章分析的最大流问题是最小费用网络流问题（Minimum-cost Network Flow Problem，MCNFP）的一个特例，对于更为一般的网络流问题，已有多种求解算法。2.3.3 节所讲的线性规划是较为通用的方法，接下来介绍另外一种求解网络流问题的方法：Ford-Fulkerson 算法（Edmonds and Karp, 1972）。

首先定义两个重要工具：残存网络（Residual Network）和增广路径（Augment Path）。

残存网络指的是具有残存容量的图。残存容量用下式表示：

$$a_f(i,j) = \begin{cases} a(i,j) - f(i,j), & \text{如果 } (i,j) \in E, \text{即 } (i,j) \text{ 是前向弧} \\ f(j,i), & \text{如果 } (j,i) \in E, \text{即 } (i,j) \text{ 是后向弧} \\ 0, & \text{其他} \end{cases}$$

残存容量解释为：当两点的连边就是原网络图的边时，其残存容量等于该边的运载上限减去已有流量；当两点的连边是原图边的反向时，其残存容量等于该边的已有流量；此外其他情况下，残存容量等于 0。

增广路径指的是残存网络中从起点到终点的一条简单路径 $L$，记为 $C$。增广路径中每条边可增加流量的最大值可表示为该路径中各边残存容量的最小值：$k = \min\{a_f(i,j) : (i,j) \in C\}$。

接下来我们讲解 Ford-Fulkerson 算法：首先将流量设置为 0，生成残存网络，寻找增广路径，计算可增加流量的最小值，更新图中的流量，然后再次生成残存网络，循环上述过程，直到无法找到增广路径，当前网络中的流就是最大流。下面是 Ford-Fulkerson 算法的具体解释。

**Step 1**：找到一个初始可行解，方便起见可以直接设置每个弧的流量为 0，此时残存网络就是原图。

**Step 2**：如果与汇点相连的所有弧段都已经标记过且无法找到增广路径了，那么当前的可行流量就是网络中的最大流量；如果与汇点相连的某些弧段 $(i,\mathrm{si}),(\forall i)$ 还没被标记，那就标记该弧段和汇点，生成残存网络并从源点标记到该弧段和汇点，即寻找增广路径，记作 $C$，并跳转至 Step 3。

**Step 3**：分两种情况。

Case 1：如果 $C$ 中的弧段全部为前向弧，对于 $C$ 中的每个弧段 $(i,j)$，记 $a(i,j)$ 为该弧上可以增加的流量（满足容量约束的条件下），因此新标记出的这个线路上可增加的流量（整个网络中可增加的流量）为

$$k=\min_{(i,j)\in C}a(i,j) \tag{2.30}$$

如果 $k>0$，那么网络中可以增加 $k$ 的流量，同时又不违背容量约束和流平衡约束。

Case 2：如果 $C$ 中的弧段既包含前向弧，也包括后向弧，对于 $C$ 中的每个后向弧 $(i,j)$，记 $r(i,j)$ 为该弧上可以减少的流量，又记

$$k_1=\min_{(i,j)\in C}a(i,j),\qquad k_2=\min_{(i,j)\in C}r(i,j) \tag{2.31}$$

如果 $k_1>0$且$k_2>0$，那么前向弧中将增加流量 $\min(k_1,k_2)$，后向弧中将减少流量 $\min(k_1,k_2)$，整体上网络中从源点到汇点将增加 $\min(k_1,k_2)$。其中 $k_1$ 与 Case 1 情况一致，因为是所有弧中可增加的最小者，因此不会违背容量约束；$k_2$ 是所有可以减少流量的后向弧中的最小者，因此每个后向弧的流量都不会减成负值，同时被减掉的流量经此弧段的上游节点流去其他弧段，也就保持了流平衡约束。最终增加的流量汇入了汇点，因此流量增加了。

调整整个网络中的流量后，返回 Step 2 继续执行该算法。

下面我们用第 2.3.2 节的例子解释 Ford-Fulkerson 算法的执行过程：首先我们给定一个初始解，将所有弧段中的流量设定为 0，如图 2.7 所示。

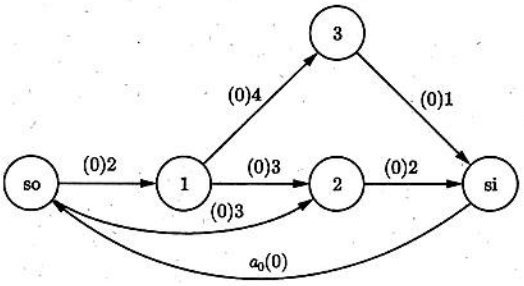



图 2.7　石油运输问题的初始解

然后进行弧段标记并进行流量调整，选取与汇点 si 相连的没有被标记的弧段 $(2,\mathrm{si})$，并从源点 so 进行标记，生成增广路径：$C=(\mathrm{so},1)-(1,2)-(2,\mathrm{si})$，$C$ 中的弧段全部为前向弧段，于是有 $k=\min\{a(\mathrm{so},1),a(1,2),a(2,\mathrm{si})\}=\min\{2,3,2\}=2$，即可以在网络中增加 2 单位流量，如图 2.8 所示。

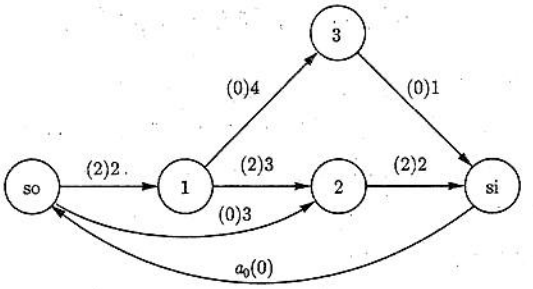



图 2.8 初始解的第一次调整

接下来继续寻找没有被标记的弧段并进行流量调整，生成残存网络，然后选取与汇点 si 相连的没有被标记的弧段 $(3,\mathrm{si})$，并从源点 so 进行标记，生成增广路径：$C=(\mathrm{so},2)-(1,2)-(1,3)-(3,\mathrm{si})$，$C$ 中的弧段既有前向弧也有后向弧，于是有 $k_1=\min\{a(\mathrm{so},2),a(1,3),a(3,\mathrm{si})\}=\min\{3,4,1\}=1$，即可以在前向弧段中增加 1 单位流量，$k_2=\min\{f(1,2)\}=\min\{2\}=2$，即可以在后向弧中减少 2 单位流量，$k=\min\{k_1,k_2\}=1$，整体来看在网络中可以增加 1 单位流量，如图 2.9 所示。

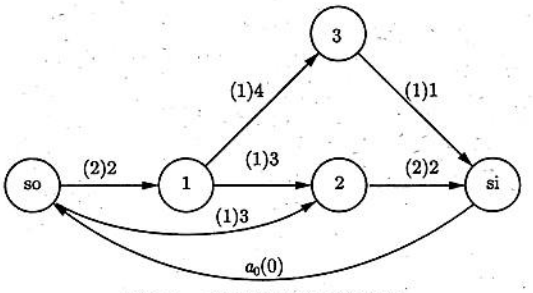



图 2.9 初始解的第二次调整

继续算法过程，发现没有弧段可以继续标记了，无法生成增广路径，结束算法，获得最优解，最大流量为 3 百万桶/时，如图 2.10 所示，与第 2.3.2 节得到的最优解一致。

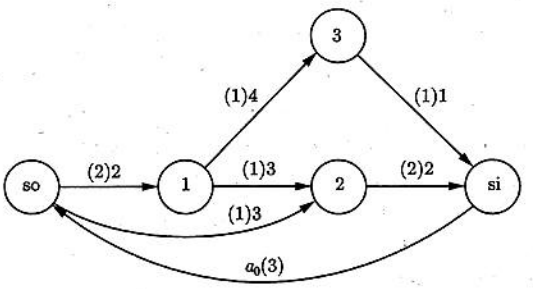



图 2.10 Ford-Fulkerson 算法求得的最优解

### 2.3.5 Java 实现 Ford-Fulkerson 算法求解最大流问题

为了方便读者更深入地理解 Ford-Fulkerson 算法，我们附上 Ford-Fulkerson 算法的 Java 实现完整代码。为了保证几个图论算法的代码通用性，我们将一些跟图的属性相关的部分单独写成类文件。算法部分都在 FordFulkerson.java 文件中。由于篇幅限制，本章只展示 FordFulkerson.java 文件的内容。

FordFulkerson.java 文件内容如下：

FordFulkerson.java

```
package maxflow;

import java.io.IOException;
import java.util.LinkedList;
import java.util.Queue;

import graph.FlowEdge;
import graph.FlowNetwork;

 /**
  * Ford-Fulkerson algorithm to compute max-flow/min-cut.
  *
  * @author Xiong Wangqi
  * @version V1.0
  * @since JDK1.8
  */
public class FordFulkerson {
    private static final double EPS = 1e-10;
    /** marked[v] = true iff s->v path in residual graph. */
    private boolean[] marked;
    /** edgeTo[v] = last edge on shortest residual s->v path. */
    private FlowEdge[] edgeTo;
    /** current value of max flow. */
    private double value;

    /**
     * Compute a max flow and min cut in the network g from vertex s to t: <br>
     * The key is increasing flow along augmenting paths.
     *
     * @param g the flow network
     * @param s the source vertex
     * @param t the sink vertex
     */
```

```
34      public FordFulkerson(FlowNetwork g, int s, int t) {
35          validate(s, g.getVertexNum());
36          validate(t, g.getVertexNum());
37
38          if (s == t) {
39              throw new IllegalArgumentException("Source equals sink");
40          }
41
42          if (!isFeasible(g, s, t)) {
43              throw new IllegalArgumentException("Initial flow is infeasible
        ");
44          }
45
46          // While there exists an augmenting path, use it(the value).
47          value = excess(g, s);
48          while (hasAugmentingPath(g, s, t)) {
49              // 1 compute bottleneck capacity
50              double bottleneck = Double.POSITIVE_INFINITY;
51              for (int v = t; v != s; v = edgeTo[v].other(v)) {
52                  bottleneck = Math.min(bottleneck, edgeTo[v].
        residualCapacityTo(v));
53              }
54
55              // 2 update the flow on the augment path
56              for (int v = t; v != s; v = edgeTo[v].other(v)) {
57                  edgeTo[v].addResidualFlowTo(v, bottleneck);
58              }
59
60              // 3 update current "max-flow"
61              value += bottleneck;
62          }
63          // If there is no augmenting path, then the max-flow/min-cut is found.
64      }
65
66  /**
67   * Is v in the s side of the min s-t cut?/Is v reachable from s in residual network?
68   *
69   * @param v the vertex
70   * @return true if vertex is on the side of mincut, false otherwise
71   */
72      public boolean inCut(int v) {
73          validate(v, marked.length);
```

```
        return marked[v];
    }

    /**
     * Return the value of the max flow.
     *
     * @return the value of the max flow
     */
    public double value() {
        return value;
    }

    /**
     * Is there an augmenting path? <br>
     * If so, upon termination edgeTo[] will contain a parent-link representation of
       such a path <br>
     * this implementation finds a shortest augmenting path (fewest number of
       edges), <br>
     * which performs well both in theory and in practice <br>
     * The augmenting path: <br>
     * 1 can increase flow on forward edges (not full) <br>
     * 2 can decrease flow on backward edge (not empty).
     *
     * @param g the flow network
     * @param s the source vertex
     * @param t the sink vertex
     * @return true if there is an augmenting path, false otherwise
     */
    private boolean hasAugmentingPath(FlowNetwork g, int s, int t) {
        int vertexNum = g.getVertexNum();
        edgeTo = new FlowEdge[vertexNum];
        marked = new boolean[vertexNum];

        // breadth-first search
        Queue<Integer> queue = new LinkedList<>();
        queue.add(s);
        marked[s] = true;
        while (!queue.isEmpty() && !marked[t]) {
            int v = queue.poll();

            for (FlowEdge e: g.adj(v)) {
                int w = e.other(v);
                if (e.residualCapacityTo(w) > 0) {
```

```
115                     if (!marked[w]) {
116                         edgeTo[w] = e;
117                         marked[w] = true;
118                         queue.add(w);
119                     }
120                 }
121             }
122         }
123
124         // Is there an augmenting path?
125         return marked[t];
126     }
127
128     private boolean isFeasible(FlowNetwork g, int s, int t) {
129         // check that capacity constraints are satisfied
130         int vertexNum = g.getVertexNum();
131         for (int v = 0; v < vertexNum; v++) {
132             for (FlowEdge e: g.adj(v)) {
133                 if (e.getFlow() < -EPS || e.getFlow() > e.getCapacity()) {
134                     System.err.println("Edge does not satisfy capacity
    constraints: " + e);
135                     return false;
136                 }
137             }
138         }
139
140         // check that net flow into a vertex equals zero, except at source and sink
141         if (Math.abs(value + excess(g, s)) > EPS) {
142             System.err.println("Excess at source = " + excess(g, s));
143             System.err.println("Max flow         = " + value);
144             return false;
145         }
146         if (Math.abs(value - excess(g, t)) > EPS) {
147             System.err.println("Excess at sink = " + excess(g, t));
148             System.err.println("Max flow         = " + value);
149             return false;
150         }
151         for (int v = 0; v < vertexNum; v++) {
152             if (v == s || v == t) {
153                 continue;
154             }
155
```

```
            if (Math.abs(excess(g, v))> EPS) {
                return false;
            }
        }

        return true;
    }

    /**
     * Return excess flow(in flow - out flow) at vertex v.
     *
     * @param g the flow network
     * @param v the vertex
     * @return excess flow at vertex v
     */
    private double excess(FlowNetwork g, int v) {
        double excess = 0.0;
        for (FlowEdge e: g.adj(v)) {
            if (v == e.from()) {
                // out-flow
                excess -= e.getFlow();
            } else {
                // in-flow
                excess += e.getFlow();
            }
        }

        return excess;
    }

    private void validate(int v, int n)  {
        if (v < 0 || v >= n) {
            throw new IndexOutOfBoundsException("vertex " + v + " is not
       between 0 and " + (n - 1));
        }
    }

    public static void main(String[] args) {
        // filename of the graph
        String filename = "./data/tinyFN.txt";

        FlowNetwork g = null;
```

```
197         try {
198             g = new FlowNetwork(filename);
199         } catch (IOException e) {
200             e.printStackTrace();
201         }
202
203         int s = 0;
204         int vertexNum = g.getVertexNum();
205         int t = vertexNum - 1;
206         FordFulkerson maxFlow = new FordFulkerson(g, s, t);
207
208         StringBuilder sb = new StringBuilder(String.format("Max flow from
        %d to %d:\n", s, t));
209         for (int v = 0; v < vertexNum; v++) {
210             for (FlowEdge e: g.adj(v)) {
211                 if ((v == e.from()) && e.getFlow() > 0) {
212                     sb.append(e);
213                     sb.append('\n');
214                 }
215             }
216         }
217
218         // print min-cut
219         sb.append("Min cut: ");
220         for (int v = 0; v < vertexNum; v++) {
221             if (maxFlow.inCut(v)) {
222                 sb.append(v + " ");
223             }
224         }
225         sb.append('\n');
226         sb.append("Max flow value = " +  maxFlow.value());
227
228         System.out.print(sb.toString());
229     }
230
231 }
```

## 2.4 最优整数解特性和幺模矩阵

在本章最短路问题的介绍中，我们提到，最短路问题（Shortest Path Problem，SPP）具有整数最优解特性，将模型的 0-1 变量约束松弛掉得到的线性松弛的最优解仍然是整数。

不仅是最短路问题，指派问题（Assignment Problem，AP）也具有相同的特性。本节我们就对这一特性进行稍微深入的解释。

首先我们来看指派问题的数学模型：

$$\min \sum_{i\in R}\sum_{j\in D} x_{ij}c_{ij} \tag{AP}$$

$$\text{s.t.} \sum_{j\in D} x_{ij}=1, \quad \forall i\in R \tag{2.32}$$

$$\sum_{i\in R} x_{ij}=1, \quad \forall j\in D \tag{2.33}$$

$$x_{ij}\in\{0,1\}, \quad \forall i\in R, j\in D \tag{2.34}$$

然后是最短路问题的数学模型：

$$\min \sum_{e\in E} d_e x_e \tag{SPP}$$

$$\text{s.t.} \sum_{e\in \text{out}(i)} x_e - \sum_{e\in \text{in}(i)} x_e = \begin{cases} 1, & i=s \\ -1, & i=t \\ 0, & \text{其他} \end{cases} \tag{2.35}$$

$$x_e\in\{0,1\}, \quad \forall e\in E \tag{2.36}$$

我们先用 Python 调用 Gurobi 验证 AP 和 SPP 的整数最优解特性，然后给出整数最优解特性成立的条件。

### 2.4.1 指派问题的最优解特性验证

我们用 Python 调用 Gurobi 建立指派问题的 0-1 整数规划模型，代码如下。

AP.py

```
from gurobipy import *
import pandas as pd
import numpy as np
import random

# generate matching value matrix
employee_num = job_num = 5
cost_matrix = np.zeros((employee_num, job_num))
for i in range(employee_num):
    for j in range(job_num):
        random.seed(i * employee_num + j)
        cost_matrix[i][j] = round(10 * random.random() + 5, 0)

# construct model object
```

```
model = Model('Assignment_Problem')

# introduce decision variable by cycling
x = [[[] for i in range(employee_num)] for j in range(job_num)]
for i in range(employee_num):
    for j in range(job_num):
        x[i][j] = model.addVar(vtype = GRB.BINARY # decision variable type
                               ,name = "x_" + str(i) + "_" + str(j)
                               )

# objective function
obj = LinExpr(0)

for i in range(employee_num):
    for j in range(job_num):
        obj.addTerms(cost_matrix[i][j], x[i][j])

model.setObjective(obj, GRB.MINIMIZE)

# Constraint 1
for j in range(employee_num):
    expr = LinExpr(0)
    for i in range(job_num):
        expr.addTerms(1, x[i][j])
    model.addConstr(expr == 1, name="D_" + str(j))

# Constraint 2
for i in range(employee_num):
    expr = LinExpr(0)
    for j in range(job_num):
        expr.addTerms(1, x[i][j])
    model.addConstr(expr == 1, name="R_" + str(i))

# solve the constructed model
model.write('model.lp')
model.optimize()

# print optimal solution
for var in model.getVars():
    if(var.x > 0):
        print(var.varName, '\t', var.x)
print('objective : ', model.ObjVal)
```

上述代码设置了任务数量和员工数量均为 5，最优解如下。

Result

```
Root relaxation: objective 4.000000e+01, 9 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node
       Time

*    0     0               0      40.0000000   40.00000  0.00%     -    0s

Explored 0 nodes (9 simplex iterations) in 0.04 seconds
Thread count was 8 (of 8 available processors)

Solution count 2: 40 55

Optimal solution found (tolerance 1.00e-04)
Best objective 4.000000000000e+01, best bound 4.000000000000e+01, gap
      0.0000%
x_0_4    1.0
x_1_2    1.0
x_2_0    1.0
x_3_3    1.0
x_4_1    1.0
objective :  40.0
```

可以看到，上述代码将指派问题建模成为 0-1 整数规划，但是在求解日志中，我们发现整数规划在根节点就得到了最优解，并没有进行任何分支。

接下来修改上述代码，将决策变量 $x_{ij}$ 的整数约束松弛掉，变成线性规划，也就是将约束 (2.34) 松弛为

$$0 \leqslant x_{ij} \leqslant 1, \quad \forall i \in R, j \in D \tag{2.37}$$

相应地，将代码中添加变量的部分修改成如下形式：

AP.py

```
# introduce decision variable by cycling
x = [[] for i in range(employee_num)] for j in range(job_num)]
for i in range(employee_num):
    for j in range(job_num):
        x[i][j] = model.addVar(lb = 0
```

```
                                ,ub = 1
                                ,vtype = GRB.CONTINUOUS    # decision
        variable type
                                ,name = "x_" + str(i) + "_" + str(j)
                                )
```

再次求解，结果如下：

Result

```
Gurobi Optimizer version 9.0.1 build v9.0.1rc0 (win64)
Optimize a model with 10 rows, 25 columns and 50 nonzeros
Model fingerprint: 0x59b89d6e
Coefficient statistics:
  Matrix range     [1e+00, 1e+00]
  Objective range  [6e+00, 2e+01]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 1e+00]
Presolve time: 0.01s
Presolved: 10 rows, 25 columns, 50 nonzeros

Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0    3.8000000e+01   2.000000e+00   0.000000e+00      0s
       3    4.0000000e+01   0.000000e+00   0.000000e+00      0s

Solved in 3 iterations and 0.01 seconds
Optimal objective  4.000000000e+01
x_0_4     1.0
x_1_2     1.0
x_2_0     1.0
x_3_3     1.0
x_4_1     1.0
objective :  40.0
```

求解结果和将变量 $x_{ij}$ 设置成 0-1 变量完全相同。可以尝试多个不同的算例，会发现不论是哪个算例，上述两种不同的决策变量设置，得到的最优解都是相同的。

### 2.4.2 最短路问题的整数最优解特性验证

我们继续用第 2.2 节的例子作为测试算例测试最短路问题（Shortest Path Problem）的最优整数特性。

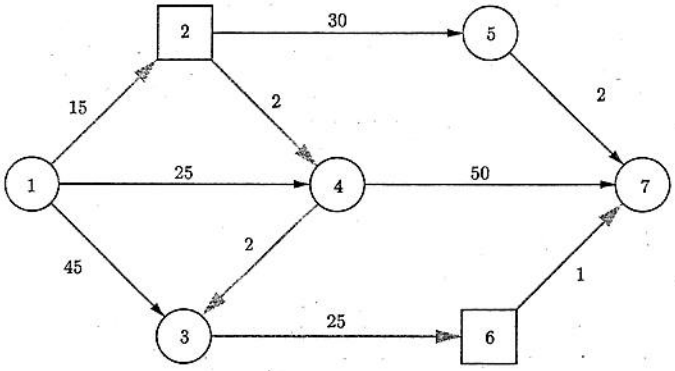



图 2.11　最短路问题：网络图

使用 Python 调用 Gurobi 建立最短路问题的 0-1 整数规划模型。具体代码如下：

SPPExample.py

```
from gurobipy import *
import pandas as pd
import numpy as np

Nodes = ['1', '2', '3', '4', '5', '6', '7']

Arcs = {('1','2'): 15
        ,('1','4'): 25
        ,('1','3'): 45
        ,('2','5'): 30
        ,('2','4'): 2
        ,('4','7'): 50
        ,('4','3'): 2
        ,('3','6'): 25
        ,('5','7'): 2
        ,('6','7'): 1
       }

model = Model('dual problem')

# add decision variables
X = {}
for key in Arcs.keys():
    index = 'x_' + key[0] + ',' + key[1]
    X[key] = model.addVar(vtype=GRB.BINARY
                          , name= index
                         )
```

```
28
29 # add objective function
30 obj = LinExpr(0)
31 for key in Arcs.keys():
32     obj.addTerms(Arcs[key], X[key])
33
34 model.setObjective(obj, GRB.MINIMIZE)
35
36 # constraint1 1 and constraint 2
37 lhs_1 = LinExpr(0)
38 lhs_2 = LinExpr(0)
39 for key in Arcs.keys():
40     if(key[0] == '1'):
41         lhs_1.addTerms(1, X[key])
42     elif(key[1] == '7'):
43         lhs_2.addTerms(1, X[key])
44 model.addConstr(lhs_1 == 1, name = 'start flow')
45 model.addConstr(lhs_2 == 1, name = 'end flow')
46
47 # constraints 3
48 for node in Nodes:
49     lhs = LinExpr(0)
50     if(node != '1' and node != '7'):
51         for key in Arcs.keys():
52             if(key[1] == node):
53                 lhs.addTerms(1, X[key])
54             elif(key[0] == node):
55                 lhs.addTerms(-1, X[key])
56     model.addConstr(lhs == 0, name = 'flow conservation')
57
58 model.write('model_spp.lp')
59 model.optimize()
60
61 print('Objective : ', model.ObjVal)
62 for var in model.getVars():
63     if(var.x > 0):
64         print(var.varName, '\t', var.x)
```

最优解如下：

Result

```
1 Variable types: 0 continuous, 10 integer (10 binary)
2 Coefficient statistics:
```

```
  Matrix range     [1e+00, 1e+00]
  Objective range  [1e+00, 5e+01]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 1e+00]
Presolve removed 9 rows and 10 columns
Presolve time: 0.00s
Presolve: All rows and columns removed

Explored 0 nodes (0 simplex iterations) in 0.01 seconds
Thread count was 1 (of 8 available processors)

Solution count 1: 45

Optimal solution found (tolerance 1.00e-04)
Best objective 4.500000000000e+01, best bound 4.500000000000e+01, gap
      0.0000%
Objective :  45.0
x_1,2    1.0
x_2,4    1.0
x_4,3    1.0
x_3,6    1.0
x_6,7    1.0
```

可以看到并没有分支，最优解为整数解。我们将变量的整数约束松弛，也就是将约束(2.36)松弛为

$$0 \leqslant x_e \leqslant 1, \quad \forall e \in E \tag{2.38}$$

在代码中将添加变量的部分修改如下：

SPP.py

```
# add decision variables
X = {}
for key in Arcs.keys():
    index = 'x_' + key[0] + ',' + key[1]
    X[key] = model.addVar(lb = 0
                          ,ub = 1
                          ,vtype = GRB.CONTINUOUS # decision variable type
                          ,name = index
                          )
```

再次求解，结果如下：

Result

```
Coefficient statistics:
  Matrix range     [1e+00, 1e+00]
  Objective range  [1e+00, 5e+01]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 1e+00]
Presolve removed 6 rows and 7 columns
Presolve time: 0.01s
Presolved: 3 rows, 3 columns, 6 nonzeros

Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0    4.3000000e+01   2.000000e+00   0.000000e+00      0s
       1    4.5000000e+01   0.000000e+00   0.000000e+00      0s

Solved in 1 iterations and 0.02 seconds
Optimal objective  4.500000000e+01
Objective :  45.0
x_1,2    1.0
x_2,4    1.0
x_4,3    1.0
x_3,6    1.0
x_6,7    1.0
```

松弛后的线性规划的解和原来整数规划的解也是完全相同的。同样地，测试多个其他算例，发现也都符合这一结果。

### 2.4.3 最优整数解特性的理解

最短路问题和指派问题是满足最优解特性的，也就是其线性松弛模型的最优解一定同时也是整数解。下面详细解释其中的原因。

考虑下面的具有 2 个决策变量的整数规划：

$$\begin{aligned} \max \quad & z = 2x_1 + x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leqslant 4 \\ & x_1 \leqslant 2 \\ & x_2 \leqslant 4 \\ & x_1, x_2 \in \mathbb{Z} \end{aligned} \tag{2.39}$$

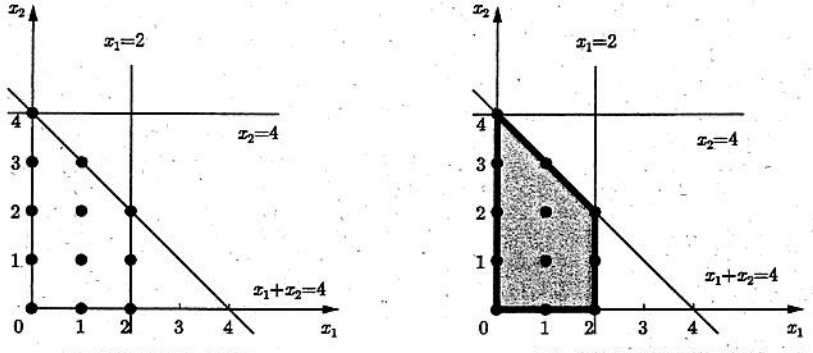
及其线性松弛问题：

$$\max \quad z = 2x_1 + x_2$$

$$\begin{aligned} \text{s.t.}\quad & x_1 + x_2 \leqslant 4 \\ & x_1 \leqslant 2 \\ & x_2 \leqslant 4 \\ & x_1, x_2 \geqslant 0 \end{aligned} \tag{2.40}$$

我们在坐标系中分别画出这两个问题的可行解和可行域，如图 2.12 所示。

容易观察到，整数规划的所有可行解围成的凸包与线性松弛问题的可行域完全相同。因此，即使我们将整数规划的整数约束松弛掉，仍然可以得到整数最优解。首先，线性规划的最优解一定是出现在极点上，并且此时线性规划的可行域等价于整数规划的可行解围成的凸包。所以，线性规划可行域的所有极点，都一定是整数规划的可行解。综合上面几点，就很容易得出，如果一个整数规划问题的所有可行解构成的凸包正好等价于其线性松弛问题的可行域，则线性松弛问题的最优解一定也同时是整数规划的最优解。




(a) 整数规划的可行解　　(b) 线性松弛问题的可行域

图 2.12　整数规划可行解围成的凸包

### 2.4.4　幺模矩阵和整数最优解特性

最短路问题和指派问题具有整数最优解特性，这是已经被证明的。但是对于任意一个整数规划，我们如何去判断该模型是否具有整数最优解特性呢？

在探讨这个问题之前，我们首先给出一些相关概念。本节内容参考自文献（陈景良，陈向晖, 2010；Shapiro，1979）。

**定义 2.4.1　整数矩阵（Integer Matrix）**：如果矩阵 $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ 的所有元素都是整数，则矩阵 $\boldsymbol{A}$ 被称为整数矩阵。

**定义 2.4.2　幺模矩阵（Unimodular Matrix）**：如果矩阵 $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ 是整数矩阵，有 $r = \operatorname{rank} \boldsymbol{A} = \min\{m, n\}$，而且 $\boldsymbol{A}$ 的所有非零 $r \times r$ 子式等于 1 或 $-1$，则 $\boldsymbol{A}$ 被称为幺模矩阵。

**定义 2.4.3　全幺模矩阵（Totally Unimodular Matrix）**：如果 $\boldsymbol{A}$ 是幺模矩阵，而且其各阶子式均等于 0、1 或 $-1$，则称 $\boldsymbol{A}$ 为全幺模矩阵。

特别地，当 $m=n$ 时，如果 $\det \boldsymbol{A}=1$ 或者 $\det \boldsymbol{A}=-1$，那么整数矩阵 $\boldsymbol{A}$ 是幺模矩阵。

不难得出下面的结论。

(1) 全幺模矩阵的所有元素均为 0、1 或 $-1$；

(2) 两个维度为 $n\times n$ 的幺模矩阵之积仍是幺模矩阵。

接下来是 3 个非常重要的定理。

**定理 2.4.1** 设 $m\leqslant n$，则对于任何整数向量 $\boldsymbol{b}$，方程组

$$\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b} \tag{2.41}$$

的所有基本解为整数向量的充分必要条件是 $\boldsymbol{A}$ 为幺模矩阵。

**定理 2.4.2** 设 $\boldsymbol{A}\in\mathbb{R}^{m\times n}$ 是整数矩阵，如果对于任何整数向量 $\boldsymbol{b}$，线性不等式组

$$\boldsymbol{A}\boldsymbol{x}\leqslant\boldsymbol{b} \tag{2.42}$$

的所有基本解是整数向量，那么 $\boldsymbol{A}$ 必是全幺模矩阵。

**定理 2.4.3** 设 $\boldsymbol{A}=[b_{ij}]\in\mathbb{R}^{m\times n}$ 是全幺模矩阵，则对 $\boldsymbol{A}$ 执行如下任一操作均不改变全幺模性。

(1) 行置换或列置换。

(2) 转置。

(3) 任一行或列乘以 $-1$。

(4) 增添只有一个元素或 1 或 $-1$ 的行和列。

上述定理阐明了，如果线性方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 或者不等式组 $\boldsymbol{A}\boldsymbol{x}\leqslant\boldsymbol{b}$ 的约束系数矩阵 $\boldsymbol{A}$ 在满足特定条件的情况下，方程组的基本解向量是整数向量，也就是方程组的解是整数解。

下面以 3 个任务和 3 个员工的指派问题为例，写出该模型的具体形式为

$$\begin{aligned}
\min\quad & z=x_{11}+x_{12}+x_{13}+x_{21}+x_{22}+x_{23}+x_{31}+x_{32}+x_{33}\\
\text{s.t.}\quad & x_{11}+x_{12}+x_{13}=1\\
& x_{21}+x_{22}+x_{23}=1\\
& x_{31}+x_{32}+x_{33}=1\\
& x_{11}+x_{21}+x_{31}=1\\
& x_{12}+x_{22}+x_{32}=1\\
& x_{13}+x_{23}+x_{33}=1\\
& 0\leqslant x_{11},\ x_{12},\ x_{13},\ x_{21},\ x_{22},\ x_{23},\ x_{31},\ x_{32},\ x_{33}\leqslant 1
\end{aligned}$$

我们提取出该模型的约束矩阵为

$$A=\begin{bmatrix}1&1&1&0&0&0&0&0&0\\0&0&0&1&1&1&0&0&0\\0&0&0&0&0&0&1&1&1\\1&0&0&1&0&0&1&0&0\\0&1&0&0&1&0&0&1&0\\0&0&1&0&0&1&0&0&1\end{bmatrix}$$

根据幺模矩阵的定义，我们取 $r=\text{rank}\ A=\min\{m,n\}=\min\{6,9\}=6$，通过检验矩阵 $A$ 所有非 0 的 $6\times 6$ 子式，无一例外，全部是 1 或者 $-1$，因此 $A$ 为幺模矩阵。所以指派问题具有整数最优解特性。

当然，指派问题和最短路问题并不是 NP-hard 问题，可以用已有的多项式时间算法很快地求解，如指派问题可以用匈牙利算法、Ford-Fulkerson 算法和 Hopcroft-Karp 算法求解。最短路问题可以用 Dijkstra 等算法求解。但是这些问题也可以通过调用求解器构建模型，直接将整数规划松弛成线性规划，从而让求解器调用单纯形法很快地求解，非常大规模的算例也可以很快地得到最优解。

## 2.5 多商品网络流问题

多商品网络流（Multicommodity Network Flow，MCNF）问题，或多物网络流问题，是指在一个网络中，多种商品从各自源点经过网络流向各自终点的网络流问题。该问题在实际生产生活中有着广泛的应用场景，如服务网络设计、通信网络、物流网络设计等。

MCNF 问题可以定义为下面的形式。

给定网络 $G=(V,A)$ 以及下面的参数：

$K$：表示商品流的集合，商品流的个数为 $|K|$；$|K|$ 个商品流的起点-终点-需求元组对分别为 $(s_1,t_1,d_1),(s_2,t_2,d_2),\cdots,(s_k,t_k,d_k)$；

$d_k$：商品流 $k\in K$ 的需求量，也就是需要从 $s_k$ 运往 $t_k$ 点的量；

$u_{ij}$：弧段 $(i,j)\in A$ 的容量，所有流经该弧段的流量总和不得超过该容量；

$c_{ij}^k$：在弧段 $(i,j)\in A$ 上运送单位商品 $k$ 所需要的成本。

MCNF 的目标就是为每个商品流设计出最优的路径，使得其从起始点到达相应的终点满足所有的需求，并且使得产生的总成本最小。

为了更直观地理解 MCNF 问题，我们使用文献（Cappanera and Scaparra, 2011）中的例子作为示例网络来引入我们的介绍。示例网络如图 2.13 所示。